

Welcome to **Zenva's Craft a Mario-Style Platformer in Phaser 3** course. Throughout this course, we will make a simple platformer where we jump over barrels and climb to the top of the screen. You will be expected to know the basics of working with the **Phaser 3 framework** and how to set up a development environment. In this lesson, we will focus on setting up our basic arcade physics for the game.

Project Setup Overview

To follow along more easily in this course, you'll want to **download** the **Course Files**. In these files, you can locate a **folder** called **00 - Starting** and open it up in a code editor of your choice. We will be using **Atom** (<https://atom.io/>) for our lessons. Regardless, in this **folder** you will find all the materials you need, including **sprite assets**, the **index.html** file that includes the **Phaser library**, and the **js files** needed to use **Phaser**.

If you take a look at **main.js**, you will find a lot of things already setup for you, including the initialization of the **scene** and the loading of your **assets**. Further, the **sprite sheets** are also already set up with the necessary **cell size** that each **frame** takes up. A bit further down, you will see that the **configuration** is already set up as well, so you can even load up the game in the browser and see it successfully loaded.

Note that you'll want to size your browser window to be in a portrait ratio - tall and narrow - in order to view and play the game properly. Refer to the browser window in the lesson video for a good reference.

Setting up Arcade Physics

Now that we've seen what's in our project setup, let's start creating our actual game. With our game, we're going to use the **Arcade Physics system** which is built into the **Phaser framework**. This is the easiest of the three **physics systems** that **Phaser** has, as it makes a lot of assumptions about how bodies are simulated.

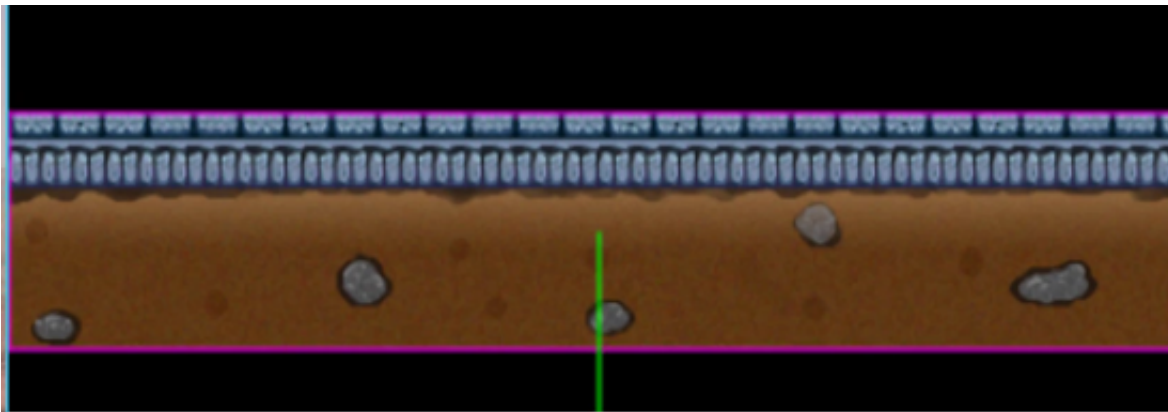
In order to use the **Arcade Physics system**, we need to include it as part of our **configuration** by declaring **physics** in **main.js**. With **physics**, we can pass in **object** that uses the **default property** to declare we're using the **arcade physics**. After this, though, we also need to create another **object** for **arcade** that passes in a **gravity value**. This determines how fast objects fall to the ground. You are free to experiment with the **values**, but know that you can use negative **values** to make objects float up and even apply **gravity** on the **x-axis**. Besides **gravity**, though, we also want to add a **debug property** so we can receive more information about the **physical bodies** we'll be creating.

```
let config = {
  type: Phaser.AUTO,
  width: 360,
  height: 640,
  scene: gameScene,
  title: 'Monster Kong',
  pixelArt: false,
  physics: {
    default: 'arcade',
    arcade: {
      gravity: {y: 1000},
      debug: true
    }
  }
};
```

Now we need to add **sprites** to our **physics system** within the **gameScene.create function**. We have two ways of doing this, which we'll demonstrate using the **ground sprite**. The first way is to create a **variable** that **adds** the **ground sprite** at an **x** and **y position**. After doing this, you can then do **this.physics.add** to add the **existing ground sprite** to the **physics system**.

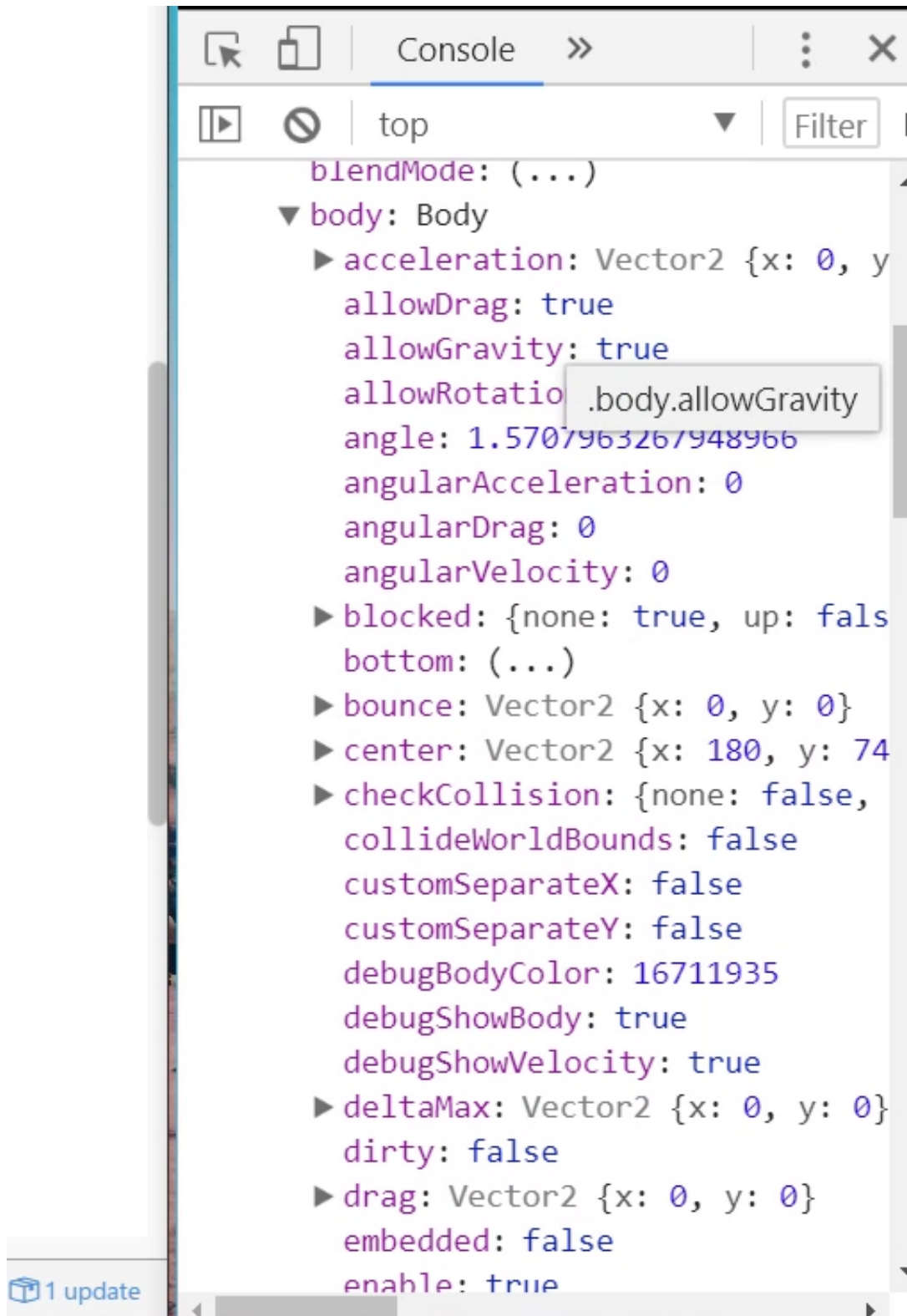
```
gameScene.create = function() {  
  
    // 1) adding existing sprites to the physics system  
    // sprite creation  
    let ground = this.add.sprite(180, 400, 'ground');  
  
    // add sprite to the physics system  
    this.physics.add.existing(ground);  
  
};
```

If you test the game out in the browser, you will see the **ground** falls off screen the minute the game loads up due to the **physics**. You may also notice a pink border surrounding the **sprite** and a **green arrow**. These come from our **debug property** we set up, as the **pink** shows your **physical body** and the **green arrow** shows the **body's velocity**.



We can also add a **console log** to our **function** so we can view the **body** information in the **console**. Under the **body property dropdown**, you will find a ton of information that has to do with the **2D physics simulation**. As you might see, there are many **properties** we can edit to our needs, such as turning off gravity for a specific sprite or changing the friction.

```
gameScene.create = function() {  
  
    // 1) adding existing sprites to the physics system  
    // sprite creation  
    let ground = this.add.sprite(180, 400, 'ground');  
  
    // add sprite to the physics system  
    this.physics.add.existing(ground);  
  
    console.log(ground);  
};
```



As mentioned, there is a second way that we can add a **sprite** to our **physics system**. Unlike how we did it before, this way requires only a single line, as we'll **add** both the **sprite** and **physics** at the same time.

```
gameScene.create = function() {  
  
    // 1) adding existing sprites to the physics system
```



```
// sprite creation
let ground = this.add.sprite(180, 400, 'ground');

// add sprite to the physics system
this.physics.add.existing(ground);

// 2) creating and adding sprites to the physics system
let ground2 = this.physics.add.sprite(180, 200, 'ground');

console.log(ground2);
};
```

If you look in the **console** for this particular **sprite**, you will notice it still has access to all the related **properties** for **physics simulation**. However, you may note that it is listed as an **arcade sprite** instead of a regular **sprite** as before. This is a minute difference, so you are free to use the method that you find more comfortable.

You are encouraged to explore what the **body** has to offer **properties** wise, but in the next lesson we will set up our **static** and **dynamic bodies** to our game. For now, you can learn more about the **Arcade Physics system** by reviewing the **Phaser 3 documentation**:

<https://photonstorm.github.io/phaser3-docs/Phaser.Physics.Arcade.ArcadePhysics.html>

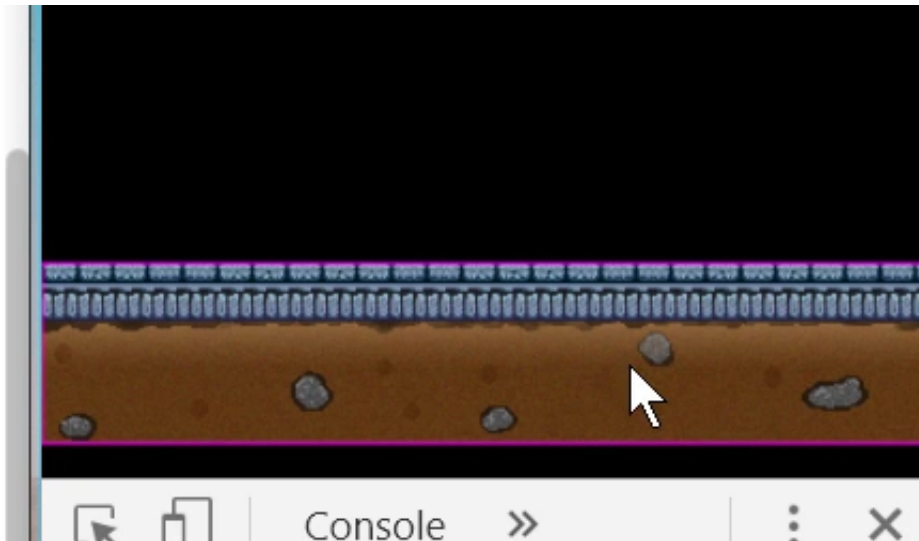
In this lesson, we're going to set up our **static** and **dynamic bodies**.

Static Objects and Collisions

At current, both our **ground sprites** fall off to oblivion since both have **gravity** acting upon them. Obviously we don't want this for our game, so we need to change this **property** from our **body** element. We can do this easily by setting **allowGravity** to be **false** for our first **ground object**.

```
gameScene.create = function() {  
  
    // ground  
    let ground = this.add.sprite(180, 400, 'ground');  
    this.physics.add.existing(ground);  
  
    // disable gravity  
    ground.body.allowGravity = false;  
  
    // 2) creating and adding sprites to the physics system  
    let ground2 = this.physics.add.sprite(180, 200, 'ground');  
};
```

When we run the game after this change, the **ground** will no longer be affected by **gravity**.



However, our **ground2 object** falls right through the **ground**. What if we wanted them to **collide**? To do this, we need to set up some **collision detection**. Using the **collider method** from adding **physics**, we can specify two things we want to **collide**. Not only can we use individual **sprites** for this, but we can also use **sprite groups**. In our case, though, we will pass in both **ground sprites**.

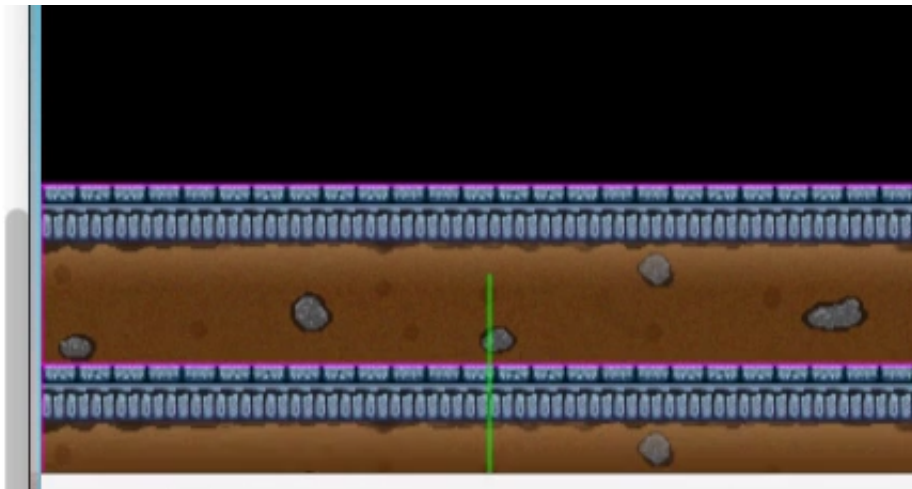
```
gameScene.create = function() {  
  
    // ground  
    let ground = this.add.sprite(180, 400, 'ground');  
    this.physics.add.existing(ground);  
  
    // disable gravity
```

```
ground.body.allowGravity = false;

// 2) creating and adding sprites to the physics system
let ground2 = this.physics.add.sprite(180, 200, 'ground');

// collision detection
this.physics.add.collider(ground, ground2);
};
```

When we do this, however, our **ground2 block** gives its **momentum** to the **ground block**, causing them both to fall.



To fix this, we can access another **property** from **body** called **immovable** and set it to **true** for the **ground sprite**. As the name implies, this prevents the object from taking on another object's momentum.

```
gameScene.create = function() {

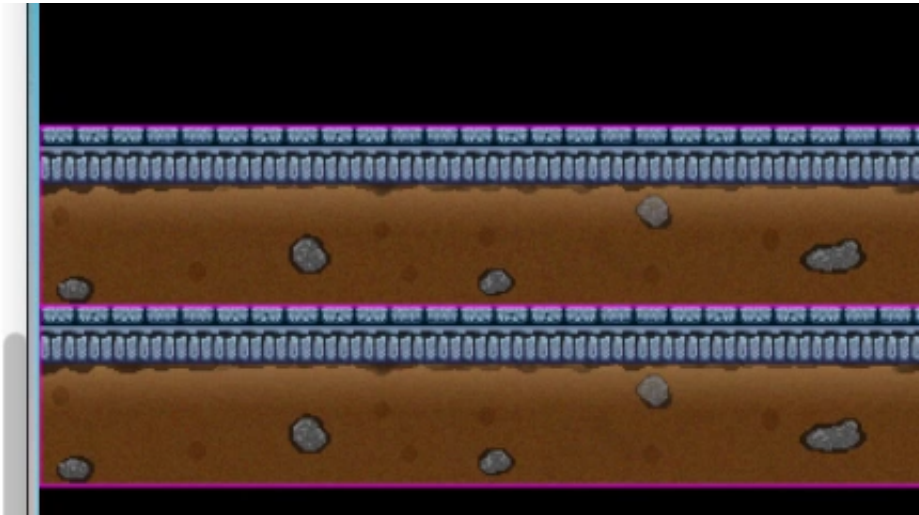
    // ground
    let ground = this.add.sprite(180, 400, 'ground');
    this.physics.add.existing(ground);

    // disable gravity
    ground.body.allowGravity = false;

    // make it immovable
    ground.body.immovable = true;

    // 2) creating and adding sprites to the physics system
    let ground2 = this.physics.add.sprite(180, 200, 'ground');

    // collision detection
    this.physics.add.collider(ground, ground2);
};
```



So we can make **immovable objects**, but what if we had a **platform** that we didn't want to receive **momentum** but wanted to move manually? **Immovable** would do us no good, so instead we can set the **sprite**, like our **ground** object, to have a **static body**. When we **add** the **sprite** to our **physics** system with our first method, there is an optional second **parameter** that is **false** by default. By setting it to **true**, we actually pass a **static body** to that **sprite object**, meaning it won't respond to any forces. After setting this to **true**, we can even **comment out** our previous adjustments besides collision, since the **static body** will not react to **gravity** or momentum.

```
gameScene.create = function() {  
  
    // ground  
    let ground = this.add.sprite(180, 400, 'ground');  
    this.physics.add.existing(ground, true);  
  
    // disable gravity  
    // ground.body.allowGravity = false;  
  
    // make it immovable  
    // ground.body.immovable = true;  
  
    // 2) creating and adding sprites to the physics system  
    let ground2 = this.physics.add.sprite(180, 200, 'ground');  
  
    // collision detection  
    this.physics.add.collider(ground, ground2);  
};
```

If you log both our **ground sprites** to the **console**, you can observe the many differences between them, such as the **ground** now specifically being a **Static body** and the **ground2 sprite** just being a **body**. The **Static body ground sprite** will also have less **properties** available to it in the **console**. You may also note while testing that instead of a **pink border**, the **ground** has a **blue border** to indicate its **static nature**.



Setting objects to **static** as we've done can be helpful to **performance**, as **Phaser** will no longer have to calculate forces for the **static bodies**. Thus, it's a fast and clean way to disable **force-related physics** in certain situations.

Platforms and Player

For our game, we're going to have both **platforms** and the **ground** be **static bodies** since they



won't move. Thus, we'll add those now. First, we'll want to **comment out** our **ground2 sprite** since we'll no longer be using it. After which, we'll also want to move our **ground sprite** to its final position on the **y-axis**.

```
gameScene.create = function() {  
  
    // ground  
    let ground = this.add.sprite(180, 604, 'ground');  
    this.physics.add.existing(ground, true);  
  
    // disable gravity  
    // ground.body.allowGravity = false;  
  
    // make it immovable  
    // ground.body.immovable = true;  
  
    // 2) creating and adding sprites to the physics system  
    // let ground2 = this.physics.add.sprite(180, 200, 'ground');  
  
    // collision detection  
    // this.physics.add.collider(ground, ground2);  
};
```

Following this, we will then create a **platform** using the first method for adding **physics** we learned last lesson. While we do have a **platform sprite**, we want to have **platforms** of different length. Thus, we will actually use our **block sprite** for it. When **adding** the **sprite**, we will use the **tileSprite method** which allows us to repeat the same texture multiple times. Like the regular **sprite method**, we have to specify the position and **sprite** we'll be using. To **tile** the **sprite**, though, we also have to specify the **width** and **height** by taking the original **width** and **height** of the **sprite** and multiplying it by the number of tiles we want for it.

```
gameScene.create = function() {  
  
    // ground  
    let ground = this.add.sprite(180, 604, 'ground');  
    this.physics.add.existing(ground, true);  
  
    // platform  
    let platform = this.add.tileSprite(180, 500, 4 * 36, 1 * 30, 'block');  
    this.physics.add.existing(platform, true);  
  
    // disable gravity  
    // ground.body.allowGravity = false;  
  
    // make it immovable  
    // ground.body.immovable = true;  
  
    // 2) creating and adding sprites to the physics system  
    // let ground2 = this.physics.add.sprite(180, 200, 'ground');  
  
    // collision detection  
    // this.physics.add.collider(ground, ground2);  
};
```



(**Note:** Don't forget to set the second parameter for physics to **true** so the **platforms** are **static bodies**).

Next, we will add the **player sprite** to our game, which will have a **dynamic body** since we want things like **gravity** to be able to act on the **player**. This will be created the same way we created the **ground**. However, when adding the **sprite**, we also have to specify the **frame** that our **player** starts on since it is on a **spritesheet**. Last, we want to make sure to save the **player** since we'll need this **player** for multiple things.

```
gameScene.create = function() {  
  
    // ground  
    let ground = this.add.sprite(180, 604, 'ground');  
    this.physics.add.existing(ground, true);  
  
    // platform  
    let platform = this.add.tileSprite(180, 500, 4 * 36, 1 * 30, 'block');  
    this.physics.add.existing(platform, true);  
  
    // player  
    this.player = this.add.sprite(180, 400, 'player', 3);  
    this.physics.add.existing(this.player);  
  
    // disable gravity  
    // ground.body.allowGravity = false;  
  
    // make it immovable  
    // ground.body.immovable = true;  
  
    // 2) creating and adding sprites to the physics system  
    // let ground2 = this.physics.add.sprite(180, 200, 'ground');  
  
    // collision detection  
    // this.physics.add.collider(ground, ground2);  
};
```

Collision Detection

Last, we need to set up **collision** between our **player** and **platforms/ground**. However, if we did each one individually, we'd wind up with hundreds of lines of code, which isn't very efficient. Instead, we can create a **group** at the top of this **function** called **platforms**. Then, when we create each of our **ground** or **platform sprites**, we can add that **sprite** object to our **platforms group**.

Finally, when it comes to our **collision detection line**, we can pass in our **player** and the **platforms group**. This ensures **collision** between the **player** and all members of the **platforms group**, whether they are **ground sprites** or **platform sprites**.

```
gameScene.create = function() {  
  
    this.platforms = this.add.group();
```



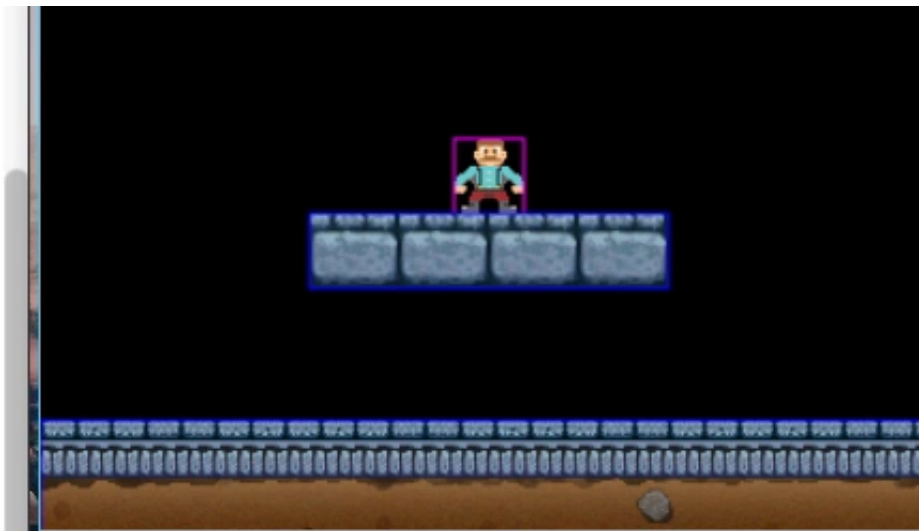
```
// ground
let ground = this.add.sprite(180, 604, 'ground');
this.physics.add.existing(ground, true);
this.platforms.add(ground);

// platform
let platform = this.add.tileSprite(180, 500, 4 * 36, 1 * 30, 'block');
this.physics.add.existing(platform, true);
this.platforms.add(platform);

// player
this.player = this.add.sprite(180, 400, 'player', 3);
this.physics.add.existing(this.player);

// collision detection
this.physics.add.collider(this.player, this.platforms);
};
```

At this point, you can test the game in the browser and see our player rest snugly on the platforms.



In the next lesson, we will set up the ability for our **player** to walk around. In the meantime, you can check out the **Phaser 3 documentation** for more information on **bodies**:

<https://photonstorm.github.io/phaser3-docs/Phaser.Physics.Arcade.Body.html>



In this lesson, we're going to get our player character walking using the arrow keys as our input.

Basic Movement

To begin, we're first going to turn to our **create function** and enable our **cursor keys** using the **input plugin** to select our **keyboard**. With these enabled, we'll be able to detect whether keys are being pressed or not and save that in the **cursors object** we just created.

```
// enable cursor keys
this.cursors = this.input.keyboard.createCursorKeys();
```

Next, we need to create a new **update method** under our **create method** which executes every frame of our game. It is in here that we'll actually check for our key presses.

```
gameScene.update = function()
{

};
```

Using an **if/else statement chain**, we can begin by going through our **cursors object** to check if the **left** or **right key** is being pressed. **If** either of those is the case, we can then **set** the **player's velocity** using the **body physics**. Since this motion represents side-to-side, we need to specifically target the **X axis**. However, if the **player** isn't moving, we need to make sure to **set** their **velocity to 0** so they don't maintain momentum.

```
gameScene.update = function()
{
    // movement to the left
    if(this.cursors.left.isDown) {
        this.player.body.setVelocityX(-100);
    }

    // movement to the right
    else if(this.cursors.right.isDown) {
        this.player.body.setVelocityX(100);
    }

    else {
        // make the player stop
        this.player.body.setVelocityX(0);
    }
};
```

(**Note:** Going to the left is a negative value for the velocity, while going to the right is a positive value).

Movement and Animations

While this gets our **player** moving, we don't have any interaction in regards to animation. Thus, we will fix that. Within the **create function** again, we will set up our actual animation right under where



we create the **player**. To **create** the **animation**, we need to specify two objects with several **properties**. This includes:

- A **key** that names what the animation is.
- A **frames property** that declares both the **spritesheet** we'll be using and the **frame numbers** that the walking animation consists of
- A **framerate** that declares how many **frames per second** this animation will play
- A **yoyo** which determines whether the animation loops back to the first frame.
- A **repeat** value which determines whether the animation repeats. For us, -1 tells the animation to repeat.

```
// walking animation
this.anims.create({
  key: 'walking',
  frames: this.anims.generateFrameNames('player', {
    frames: [0, 1, 2]
  }),
  frameRate: 12,
  yoyo: true,
  repeat: -1
});
```

(**Note:** You can learn more about animations via this **documentation**:

<https://photonstorm.github.io/phaser3-docs/Phaser.GameObjects.Components.Animation.html>)

Back in the **update function**, we can now implement our **walking animation** by setting it to **play** with each key press. However, there are two crucial things we need to do. First, before we **play** the animation, we need to check **if one isn't already playing**. If we don't do this, the character walk cycle will stop when we press the key. Second, we need to use the **flipX property** to **flip** the **sprite** so it faces the correct direction. Since the character faces **left** by default, we need to have the right direction set the **flipX value** to **true**.

```
gameScene.update = function()
{
  // movement to the left
  if(this.cursors.left.isDown) {
    this.player.body.setVelocityX(-100);

    this.player.flipX = false;

    // play animation if none is playing
    if(!this.player.anims.isPlaying)
      this.player.anims.play('walking');
  }

  // movement to the right
  else if(this.cursors.right.isDown) {
    this.player.body.setVelocityX(100);

    this.player.flipX = true;

    // play animation if none is playing
    if(!this.player.anims.isPlaying)
```



```
        this.player.anims.play('walking');
    }
    else {
        // make the player stop
        this.player.body.setVelocityX(0);
    }
};
```

Of course, we don't want the character animation to continue if we aren't moving. As such, we need to make sure to **stop** the **walking animation** and **set** the **frame** to **3** so it's back to our default idle sprite.

```
gameScene.update = function()
{
    // movement to the left
    if(this.cursors.left.isDown) {
        this.player.body.setVelocityX(-100);

        this.player.flipX = false;

        // play animation if none is playing
        if(!this.player.anims.isPlaying)
            this.player.anims.play('walking');
    }

    // movement to the right
    else if(this.cursors.right.isDown) {
        this.player.body.setVelocityX(100);

        this.player.flipX = true;

        // play animation if none is playing
        if(!this.player.anims.isPlaying)
            this.player.anims.play('walking');
    }
    else {
        // make the player stop
        this.player.body.setVelocityX(0);

        // stop walking animation
        this.player.anims.stop('walking');

        // set default frame
        this.player.setFrame(3);
    }
};
```

Player Speed

Before we end the lesson, we're going to make a small tweak. Where we **set** the **velocity**, we are



using hard coded numbers, which is not a good practice for programming. Thus, we're going to use a **variable** instead. Up in the **init function**, we're going to set up a **player speed variable**. We will also set up a **jumpSpeed** which we'll use next lesson.

```
gameScene.init = function() {  
  
    // player parameters  
    this.playerSpeed = 150;  
    this.jumpSpeed = -600;  
};
```

(Note: JumpSpeed is negative as it designates we're moving up on the Y-axis).

Then, back with our **update function**, we can plug **playerSpeed** in to set the **velocities** instead. In so doing, we only have to change the **variable** in the future if we want to change the speed.

```
// executed on every frame  
gameScene.update = function()  
{  
    // movement to the left  
    if(this.cursors.left.isDown) {  
        this.player.body.setVelocityX(-this.playerSpeed);  
  
        this.player.flipX = false;  
  
        // play animation if none is playing  
        if(!this.player.anims.isPlaying)  
            this.player.anims.play('walking');  
    }  
  
    // movement to the right  
    else if(this.cursors.right.isDown) {  
        this.player.body.setVelocityX(this.playerSpeed);  
  
        this.player.flipX = true;  
  
        // play animation if none is playing  
        if(!this.player.anims.isPlaying)  
            this.player.anims.play('walking');  
    }  
    else {  
        // make the player stop  
        this.player.body.setVelocityX(0);  
  
        // stop walking animation  
        this.player.anims.stop('walking');  
  
        // set default frame  
        this.player.setFrame(3);  
    }  
};
```




Our character is now able to walk smoothly with our animations in place.



In the next lesson, we're going to set up the character's jump ability! If you would like to read more about the **input plugin**, you can review the following **documentation**:
<https://photonstorm.github.io/phaser3-docs/Phaser.Input.InputPlugin.html>



In this lesson, we're going to implement the **player's** jumping ability and also confine the player to the world boundaries.

Constraining the Player to the World

The first thing we're going to do is constrain the **player** to the **world bounds**. Above our **player walking animation** setup in the **create function**, we can do this easily by **setting** the **CollideWorldBounds** property in the **player's body** to **true**.

```
// constrain player to the game bounds
this.player.body.setCollideWorldBounds(true);
```

However, we also need to define what the **world bounds** so the **player** can actually **collide** with those bounds. This is especially important for worlds that are larger than the screen/camera size, since left alone the player would not be able to explore the world in full. In the **create function** above our **platforms group**, we can do this by specifying the **width** and **height** going through the **physics world bounds** as seen below. In our case, we want the **height** to be larger than our screen size so we can travel further up.

```
// world bounds
this.physics.world.bounds.width = 360;
this.physics.world.bounds.height = 700;
```

Player Jumping

Now we can implement the **player jumping** functionality in the **update function**. Under our movements for the **X axis**, we can set up another **if statement** that checks if the **space or down key** is pressed. Like before, we can then **set** the **player's velocity**, this time using the **Y axis** and the **jumpSpeed**. However, like we did when the **player** isn't moving, we want to make sure to **stop** the **walking animation** and set a **frame** to use (in this case, **frame 2**).

```
// handle jumping
if(this.cursors.space.isDown || this.cursors.up.isDown) {
  // give the player a velocity in Y
  this.player.body.setVelocityY(this.jumpSpeed);

  // stop the walking animation
  this.player.anims.stop('walking');

  // change frame
  this.player.setFrame(2);
}
```

The character technically can jump with just this, but we have two problems: the jump frame doesn't stay if we let the key go, and the **player** can jump infinitely. To fix this, we need to be able to check if we're actually on the **ground** or not.

At the top of the **update function**, we can create a **variable** to store whether we're **on** the **ground** or not. To determine whether this is **true** or **false**, we can check whether the **player's**



body is **blocked** from moving down. **If** that winds up being the case, the **variable** will be set to **true**, because we'll be on the **ground** in that situation. For good measure, we'll also let the **body touching** something below it determine the **variable** as well, just in case we have things that **body blocked down** doesn't react with.

```
// are we on the ground?
let onGround = this.player.body.blocked.down ||
this.player.body.touching.down;
```

Back with our **movements**, we can now use this **variable** to alter our movements based on whether we're **on the ground** or not. So that we can't walk mid-air, we'll want to add **onGround** as part of the check for **playing** our **walk animations**. When the **player stops**, we also want to check **if** the **player** is on the **ground** before setting our **frame** to **3**. This way the **frame** doesn't change while we're mid-air. Finally, within our **jumping movement**, we want to check if we're **on the ground** when we press the **space** or **up** key. This ultimately prevents us infinitely jumping in the air.

(**Note:** All of these use **and (&&)** since we want **onGround** and our previous checks to both be true).

```
// executed on every frame
gameScene.update = function()
{
    // are we on the ground?
    let onGround = this.player.body.blocked.down ||
this.player.body.touching.down;

    // movement to the left
    if(this.cursors.left.isDown) {
        this.player.body.setVelocityX(-this.playerSpeed);

        this.player.flipX = false;

        // play animation if none is playing
        if(onGround && !this.player.anims.isPlaying)
            this.player.anims.play('walking');
    }

    // movement to the right
    else if(this.cursors.right.isDown) {
        this.player.body.setVelocityX(this.playerSpeed);

        this.player.flipX = true;

        // play animation if none is playing
        if(onGround && !this.player.anims.isPlaying)
            this.player.anims.play('walking');
    }
    else {
        // make the player stop
        this.player.body.setVelocityX(0);

        // stop walking animation
```



```
this.player.anims.stop('walking');

// set default frame
if(onGround)
    this.player.setFrame(3);
}

// handle jumping
if(onGround && (this.cursors.space.isDown || this.cursors.up.isDown)) {
    // give the player a velocity in Y
    this.player.body.setVelocityY(this.jumpSpeed);

    // stop the walking animation
    this.player.anims.stop('walking');

    // change frame
    this.player.setFrame(2);
}
};
```

The **player** can now jump into the air with ease!





In the next lesson, we will set up our level data and how to instantiate it. You can learn a bit more about how we checked being on the ground by visiting this **Phaser 3**

documentation:

https://photonstorm.github.io/phaser3-docs/Phaser.Physics.Arcade.Body.html#blocked__anchor



In this level, we're going to set up an external JSON file for our level and learn how to load it into our game.

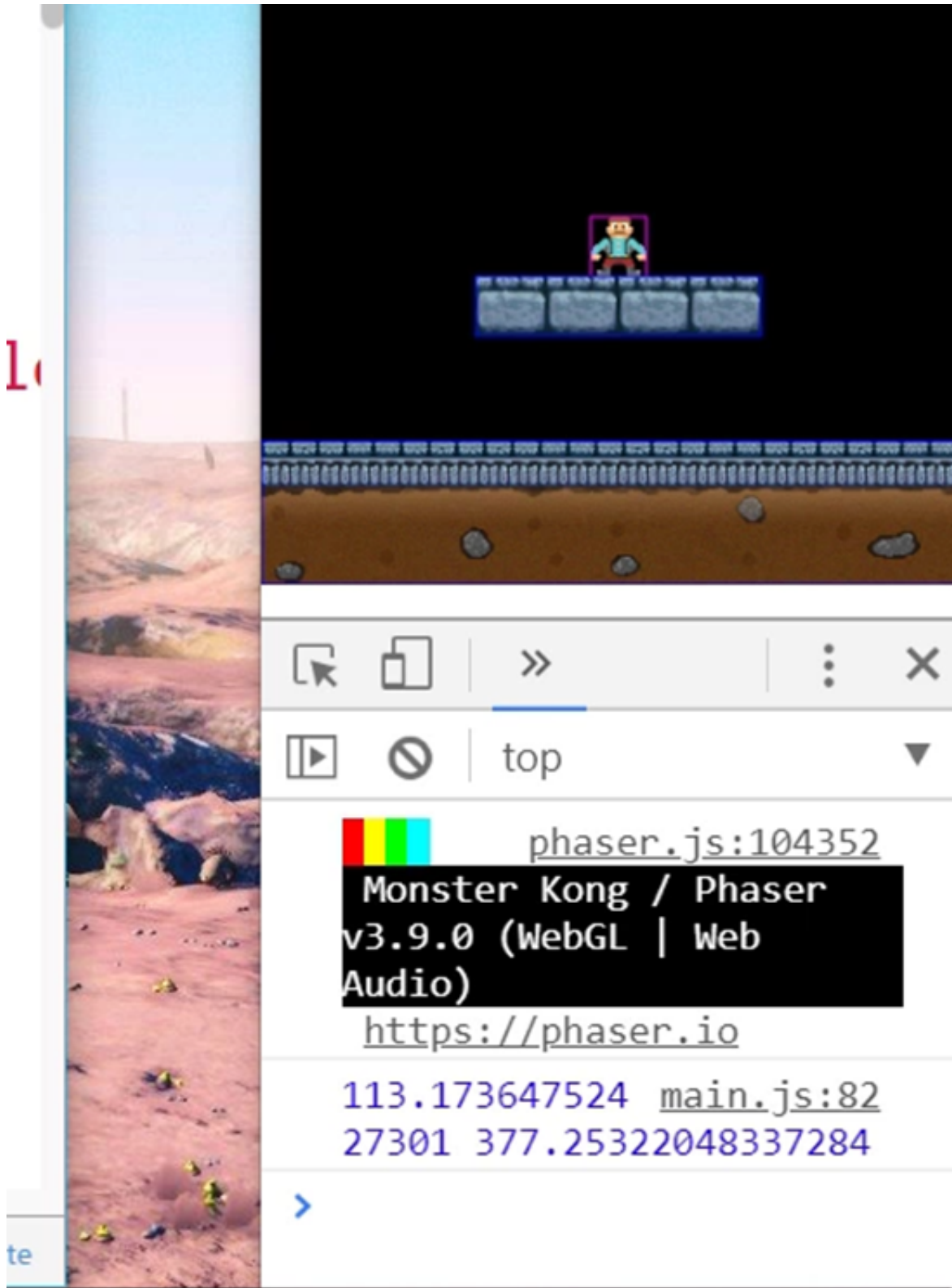
Level Design Coordinate Tip

At the moment, our current level is designed by manually adding our elements and using coordinates to determine where we'll position things. As we start to add more platforms, you can imagine the latter of these tasks would get very tedious, since we'd have to go back and forth to determine our coordinates. Rather, what would be more efficient to be able to click on our screen and have our **Console** print out what those coordinates are.

Fortunately, we can indeed do just that. Within the **create function** under our **cursors**, we can create a **callback function** that activates **on pointer down** (i.e. a click of our mouse). When this occurs, we can then **log** both our **x** and **y coordinates** to our **console**.

```
this.input.on('pointerdown', function(pointer) {  
    console.log(pointer.x, pointer.y);  
})
```

With this addition, you'll be able to click anywhere on the screen and receive the coordinates back, allowing you to more quickly adjust the numbers.



Data Model Setup

Now that we can design quickly, let's set up our actual level more. Right now, manually adding them is tedious and not suitable for having hundreds of platforms. Instead, what we want to do is create a **data model** of our **level**. Within the **init function** of **main.js**, we're going to create an **object** called **levelData** and make it an **array** of **platforms**. Within this **array** we will have various



objects, or **platforms**, that specify certain properties. For placement, we obviously want an **x** and **y** value. We'll also want to know the **number** of **tiles** our **platform** is and the **sprite** it'll be using. This will be reminiscent of how we created our **platform** before, but with a different format.

```
this.levelData = {
  "platforms": [
    {
      "x": 72,
      "y": 450,
      "numTiles": 6,
      "key": "block"
    },
    {
      "x": 0,
      "y": 330,
      "numTiles": 8,
      "key": "block"
    },
    {
      "x": 72,
      "y": 210,
      "numTiles": 8,
      "key": "block"
    },
    {
      "x": 0,
      "y": 90,
      "numTiles": 7,
      "key": "block"
    },
    {
      "x": 0,
      "y": 560,
      "numTiles": 1,
      "key": "ground"
    }
  ]
}
```

Once this **data model** is created, you can **delete** the **ground** and **platform setups** in the **create function**, as our soon-to-be external data will handle it all. Instead, we will call a **method** called **setupLevel** that we'll create next.

```
// executed once, after assets were loaded
gameScene.create = function() {

  // world bounds
  this.physics.world.bounds.width = 360;
  this.physics.world.bounds.height = 700;

  // add all level elements
  this.setupLevel();
}
```




```
// player
this.player = this.add.sprite(175, 280, 'player', 3);
this.physics.add.existing(this.player);

// constraint player to the game bounds
this.player.body.setCollideWorldBounds(true);

// walking animation
this.anims.create({
  key: 'walking',
  frames: this.anims.generateFrameNames('player', {
    frames: [0, 1, 2]
  }),
  frameRate: 12,
  yoyo: true,
  repeat: -1
});

// collision detection
this.physics.add.collider(this.player, this.platforms);

// enable cursor keys
this.cursors = this.input.keyboard.createCursorKeys();

this.input.on('pointerdown', function(pointer) {
  console.log(pointer.x, pointer.y);
});
};
```

Setup Level Method

Under our **update function**, we're going to set up a new **method** called **setupLevel**. Within this **method**, the first thing we'll do is create our **platforms group** so we can add **physics** later. After this, we need a way to **create** all our **platforms**. To do this, we can create a **for loop** that runs based on the number of **platforms** we have in our **levelData**.

```
// sets up all the elements in the level
gameScene.setupLevel = function() {
  this.platforms = this.add.group();

  // create all the platforms
  for (let i = 0; i < this.levelData.platforms.length; i++) {

  }
};
```

Within the **loop**, we'll want to store every one of our **platforms** into a **current variable** so we can access **data** on it. This can be done by simply using the **loops initial state** as the **index** of the **platforms array**. We'll also create a **variable** to store the final new **object** that will be created.



```
// sets up all the elements in the level
gameScene.setupLevel = function() {
  this.platforms = this.add.group();

  // create all the platforms
  for (let i = 0; i < this.levelData.platforms.length; i++) {
    let curr = this.levelData.platforms[i];

    let newObj;
  }
};
```

Next, we will create our actual object that will be added to the scene. Recall, though, that the **ground** uses **add sprite** while the **platforms** use **tile sprite**. In order to call the correct **function**, we can use an **if statement** to check **if** the **number of tiles** is **1**. If it is, this means we're using our **ground sprite** and can run the **add sprite function** to save it to our **newObj variable**. Since our **data model** contains the necessary **x, y**, and **key values** for the **function**, we can plug those in. However, we also need to use **set Origin** to define where our coordinate system starts. In this case, **0** is our bottom left corner.

```
// sets up all the elements in the level
gameScene.setupLevel = function() {
  this.platforms = this.add.group();

  // create all the platforms
  for (let i = 0; i < this.levelData.platforms.length; i++) {
    let curr = this.levelData.platforms[i];

    let newObj;

    // create object
    if(curr.numTiles == 1) {
      // create sprite
      newObj = this.add.sprite(curr.x, curr.y, curr.key).setOrigin(0);
    }
    else {
      // create tilesprite
    }
  }
};
```

Our **tile sprite**, which is added in the **else statement**, is a bit different. To create that, we first need to create **variables** that store the **width** and **height** of the **sprite** we'll be using. By doing this, it doesn't matter which **sprite** we use for this **method**, as it'll still be able to work even if the size of the **sprite** is different. To get those **properties**, we need to **get** the **textures** based on the **sprite** (i.e. the key in our data model) as seen below. However, we also need to specify the **frame** we want, since we could use **sprite sheets** for this as well. Last, we then can specify whether we want the **width** or **height**.



Once we have these properties, we can **add** the **tileSprite** as we did last time to our **newObj variable**, once again plugging in the **values** we saved to the **data model**. Like before, we'll also want to **set** the **Origin**.

```
gameScene.setupLevel = function() {
    this.platforms = this.add.group();

    // create all the platforms
    for (let i = 0; i < this.levelData.platforms.length; i++) {
        let curr = this.levelData.platforms[i];

        let newObj;

        // create object
        if(curr.numTiles == 1) {
            // create sprite
            newObj = this.add.sprite(curr.x, curr.y, curr.key).setOrigin(0);
        }
        else {
            // create tilesprite
            let width = this.textures.get(curr.key).get(0).width;
            let height = this.textures.get(curr.key).get(0).height;
            newObj = this.add.tileSprite(curr.x, curr.y, curr.numTiles * width, height, curr.key).setOrigin(0);
        }

    }
};
```

Last for our **loop**, we want to add **physics** to our **newObj object** and make sure to set it to **static**. After this, we can then **add** the **object** to our **platforms group** for **collision** purposes.

```
// sets up all the elements in the level
gameScene.setupLevel = function() {
    this.platforms = this.add.group();

    // create all the platforms
    for (let i = 0; i < this.levelData.platforms.length; i++) {
        let curr = this.levelData.platforms[i];

        let newObj;

        // create object
        if(curr.numTiles == 1) {
            // create sprite
            newObj = this.add.sprite(curr.x, curr.y, curr.key).setOrigin(0);
        }
        else {
            // create tilesprite
            let width = this.textures.get(curr.key).get(0).width;
            let height = this.textures.get(curr.key).get(0).height;
            newObj = this.add.tileSprite(curr.x, curr.y, curr.numTiles * width, height, curr.key).setOrigin(0);
        }

        // add physics
        newObj.setPhysics({
            type: 'static',
            x: curr.x,
            y: curr.y,
            width: curr.width,
            height: curr.height
        });

        this.platforms.add(newObj);
    }
};
```



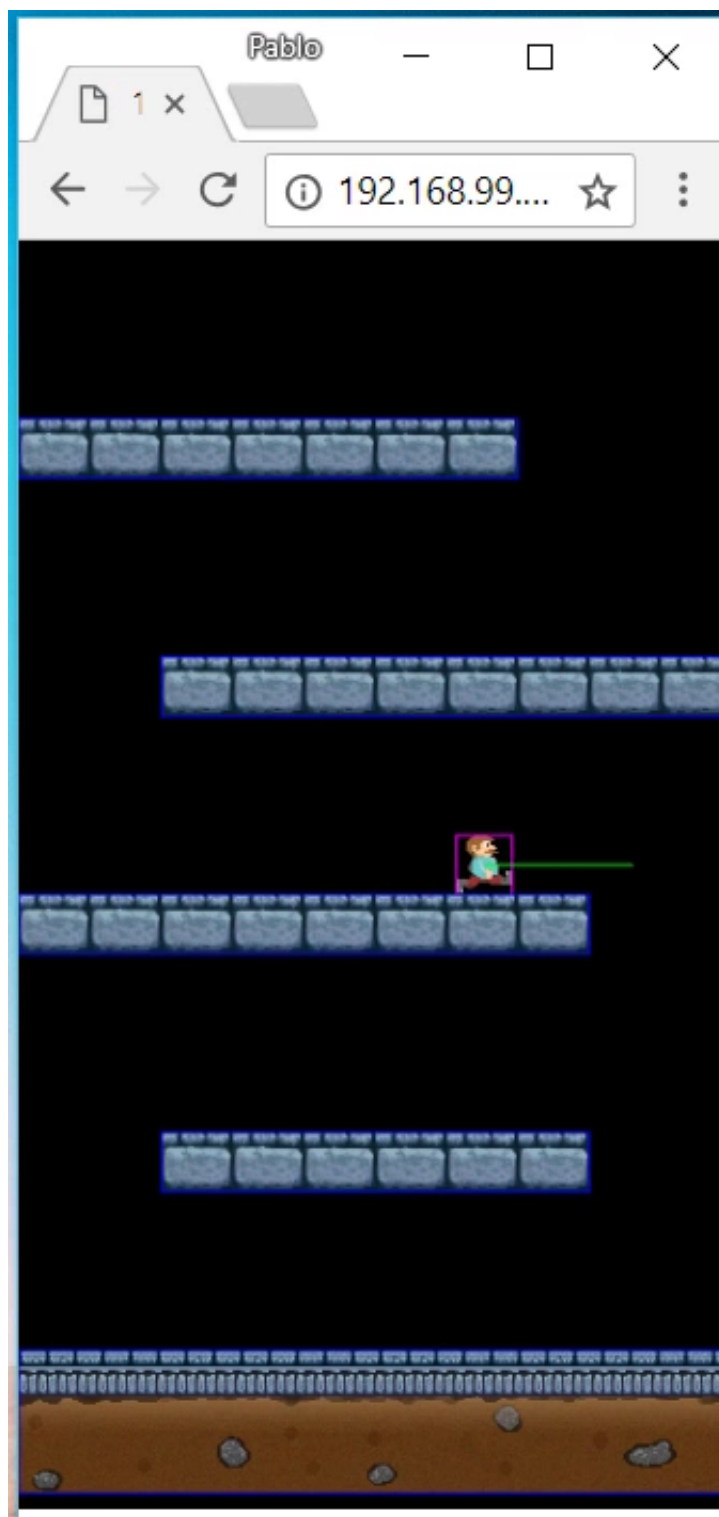
```
rr.key).setOrigin(0);
    }

    // enable physics
    this.physics.add.existing(newObj, true);

    // add to the group
    this.platforms.add(newObj);
}

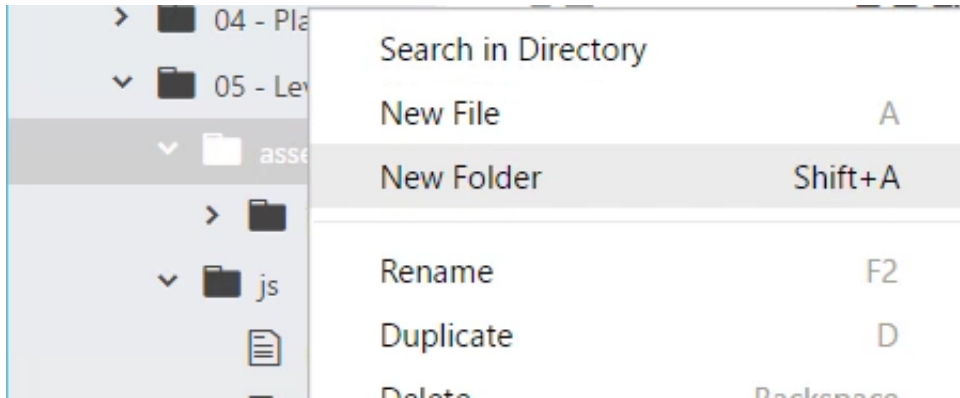
};
```

At this point, we have a functioning level again.



Externalizing the Data

Our level works, but we want to be able to load our **level Data** from an **external JSON** file instead. In so doing, it will be easier for you to update the levels without needing to change the actual code of the game. To start, we're first going to create a **New Folder** in the **assets folder** called **json**. Within the **json folder**, you'll then want to create a **new file** called **levelData.json**.



Now we can add our **JSON**. In the simplest terms, this will look just like a **Javascript object**, so for the most part we can even **copy** and **paste** our data from the **init function** in **main.js**. The key difference between the two is that every single **key** and **string** must be **double quoted**, as seen below. However, you can learn more about formatting in **JSON** by visiting the following website: <http://json.org/>

```
{
  "platforms": [
    {
      "x": 72,
      "y": 450,
      "numTiles": 6,
      "key": "block"
    },
    {
      "x": 0,
      "y": 330,
      "numTiles": 8,
      "key": "block"
    },
    {
      "x": 72,
      "y": 210,
      "numTiles": 8,
      "key": "block"
    },
    {
      "x": 0,
      "y": 90,
      "numTiles": 7,
      "key": "block"
    },
    {
      "x": 0,
      "y": 560,
      "numTiles": 1,
      "key": "ground"
    }
  ]
}
```

(**Note:** To make the process quicker, you can also visit <https://freeformatter.com/json-formatter.html> which will auto format everything for you for free).

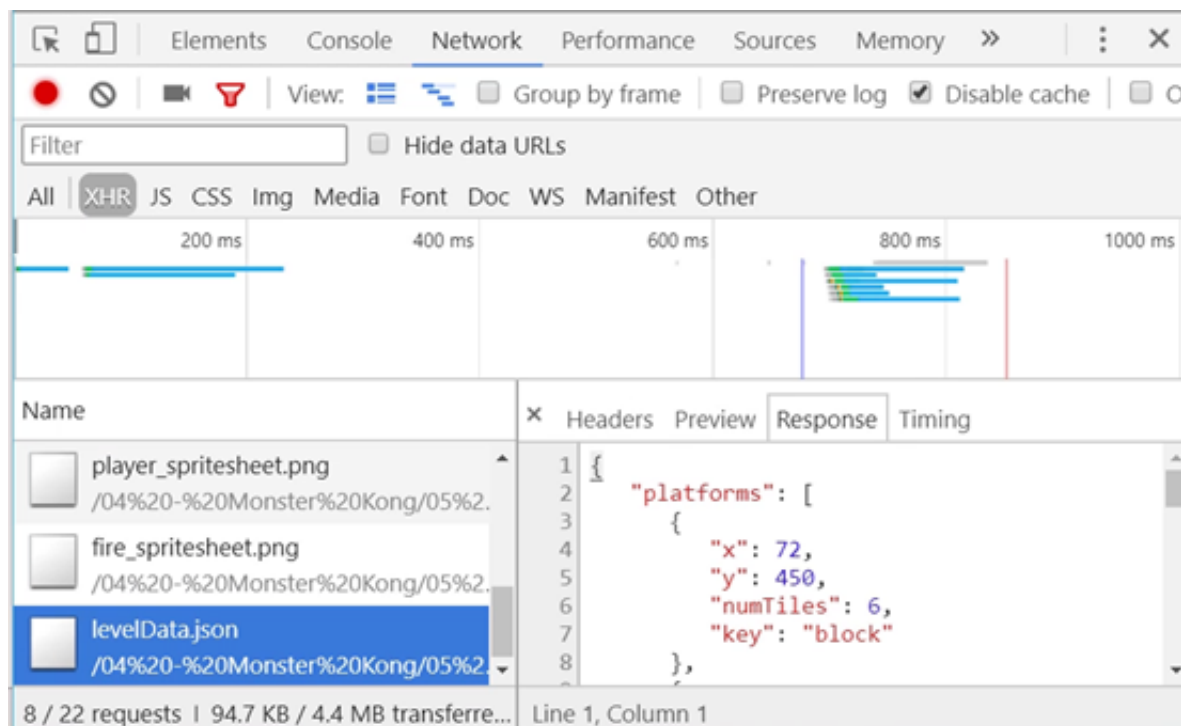
Once this is added, you can **save** the **levelData.json** file and return to **main.js**. In the **init function**, you'll want to remove the current **level Data object** since we'll no longer need it in this file. However, how do we get **setupLevel** to use our **JSON data**? First, within the **preload method**, we need to **load** in our **levelData.json** file. We'll give it a **key** of **levelData** for simplicity sake.

```
this.load.json('levelData', 'assets/json/levelData.json');
```

Then, in our **SetupLevel method**, we can **parse** this file. Above our **for loop**, we can create a new **variable** for **levelData**. Then, to this **variable**, we can **cache** our **json data** from our **levelData.json** file to use it as part of the **method**.

```
// parse json data  
this.levelData = this.cache.json.get('levelData');
```

After this, we should be loading data from an external file, which you can confirm with the **Chrome Developer Tools**.



In the next lesson, we will add the fire obstacle that is part of our game.

In this lesson, we will position some of the other things in our game using our **JSON file** from the last lesson.

Adding Positions and Fires in the JSON File

Before we add any new obstacles to our game, we first want to be able to **position** our **player** and **goal** with the **levelData.json file**. By implementing it this way, we would be able to create multiple levels with our **JSON** files and have all the elements we need be contained within it. To start, in our **JSON object**, we want to begin by adding two new **keys**: one for **player** and one for **goal**. To these **keys**, we will add **objects** that contain the **X** and **Y value** for their positions.

```
{
  "player": {
    "x": 190,
    "y": 538
  },
  "goal": {
    "x": 30,
    "y": 65
  },
  "platforms": [{
    "x": 72,
    "y": 450,
    "numTiles": 6,
    "key": "block"
  },
  {
    "x": 0,
    "y": 330,
    "numTiles": 8,
    "key": "block"
  },
  {
    "x": 72,
    "y": 210,
    "numTiles": 8,
    "key": "block"
  },
  {
    "x": 0,
    "y": 90,
    "numTiles": 7,
    "key": "block"
  },
  {
    "x": 0,
    "y": 560,
    "numTiles": 1,
    "key": "ground"
  }
]
```




After this, we can add a **key** for **fires** under our **platforms**. Like our **platforms**, this will be an **array of objects**. We don't need anything special for these **fires**, so we can simply just store their **X** and **Y coordinates**.

```
{
  "player": {
    "x": 190,
    "y": 538
  },
  "goal": {
    "x": 30,
    "y": 65
  },
  "platforms": [{
    "x": 72,
    "y": 450,
    "numTiles": 6,
    "key": "block"
  },
  {
    "x": 0,
    "y": 330,
    "numTiles": 8,
    "key": "block"
  },
  {
    "x": 72,
    "y": 210,
    "numTiles": 8,
    "key": "block"
  },
  {
    "x": 0,
    "y": 90,
    "numTiles": 7,
    "key": "block"
  },
  {
    "x": 0,
    "y": 560,
    "numTiles": 1,
    "key": "ground"
  }
],
  "fires": [
    {
      "x": 0,
      "y": 100
    },
    {
      "x": 0,
      "y": 200
    },
    {
      "x": 0,
```



```
        "y": 300
      },
      {
        "x": 0,
        "y": 400
      }
    ]
  }
}
```

Adding our Fire Elements

Now that our **JSON data** is in place, we can work with our **setupLevel method** in **main.js** again. We will begin by moving our **platforms group** to be right above our **for loop** for the **platforms** to keep things organized. Then, beneath the **for loop** for the **platforms**, we can create a new section for the **fires**. Like with **platforms**, we're going to create a **fires group** for **collision** and then make a **for loop** so we can iterate through all the **fires** in our **fires array** from the **levelData**.

```
// sets up all the elements in the level
gameScene.setupLevel = function() {

  // load json data
  this.levelData = this.cache.json.get('levelData');

  // create all the platforms
  this.platforms = this.add.group();
  for (let i = 0; i < this.levelData.platforms.length; i++) {
    let curr = this.levelData.platforms[i];

    let newObj;

    // create object
    if(curr.numTiles == 1) {
      // create sprite
      newObj = this.add.sprite(curr.x, curr.y, curr.key).setOrigin(0);
    }
    else {
      // create tilesprite
      let width = this.textures.get(curr.key).get(0).width;
      let height = this.textures.get(curr.key).get(0).height;
      newObj = this.add.tileSprite(curr.x, curr.y, curr.numTiles * width, height, curr.key).setOrigin(0);
    }

    // enable physics
    this.physics.add.existing(newObj, true);

    // add to the group
    this.platforms.add(newObj);
  }

  // create all the fire
  this.fires = this.add.group();
  for (let i = 0; i < this.levelData.fires.length; i++) {
```



```
}  
};
```

The logic for our **fires for loop** will be much the same as the **platforms**. First we will store the **fire object** we're on. Since we don't need to check for a **tile number** this time, we can immediately store our **newObj variable** with our **add sprite method** that plugs in all our **position data** and **sets** the **origin**. Since they'll all use the same **sprite**, we can manually plug in **fire**. After, we can then **add** the **physics**, which this time will be a **dynamic body**. Last, we will make sure they're added to our **fires group**.

```
// create all the fire  
this.fires = this.add.group();  
for (let i = 0; i < this.levelData.fires.length; i++) {  
    let curr = this.levelData.fires[i];  
  
    let newObj = this.add.sprite(curr.x, curr.y, 'fire').setOrigin(0);  
  
    // enable physics  
    this.physics.add.existing(newObj);  
  
    // add to the group  
    this.fires.add(newObj);  
}
```

However, there are a few special things we need to do for our **fire creation**. First off, while we do want the rest of our **dynamic body physics**, we don't want the **fires** to be **moveable** or be **affected by gravity**. Thus, we need to set those **body properties** appropriately.

```
// enable physics  
this.physics.add.existing(newObj);  
newObj.body.allowGravity = false;  
newObj.body.immovable = true;
```

Next, our **fires** will have a **sprite sheet** animation, so we need to head up to the **create method** and create the **animation** under our **player's walking animation**. For simplicity sake, we can even **copy** and **paste** the **walking animation** and just change the **values** around. Besides the **key** and **sprite name**, we need to be sure to change the **frames** used and the **frame rate to 4 frames per second**. Note that we also don't need **yoyo** in this case since there are only two frames.

However, we need to make a quick tweak. Our **setupLevel method** runs before our **animations** are created. As such, we need to take our **animations** and move them to be before the **world bounds** section.

```
// fire animation  
this.anims.create({  
    key: 'burning',  
    frames: this.anims.generateFrameNames('fire', {
```



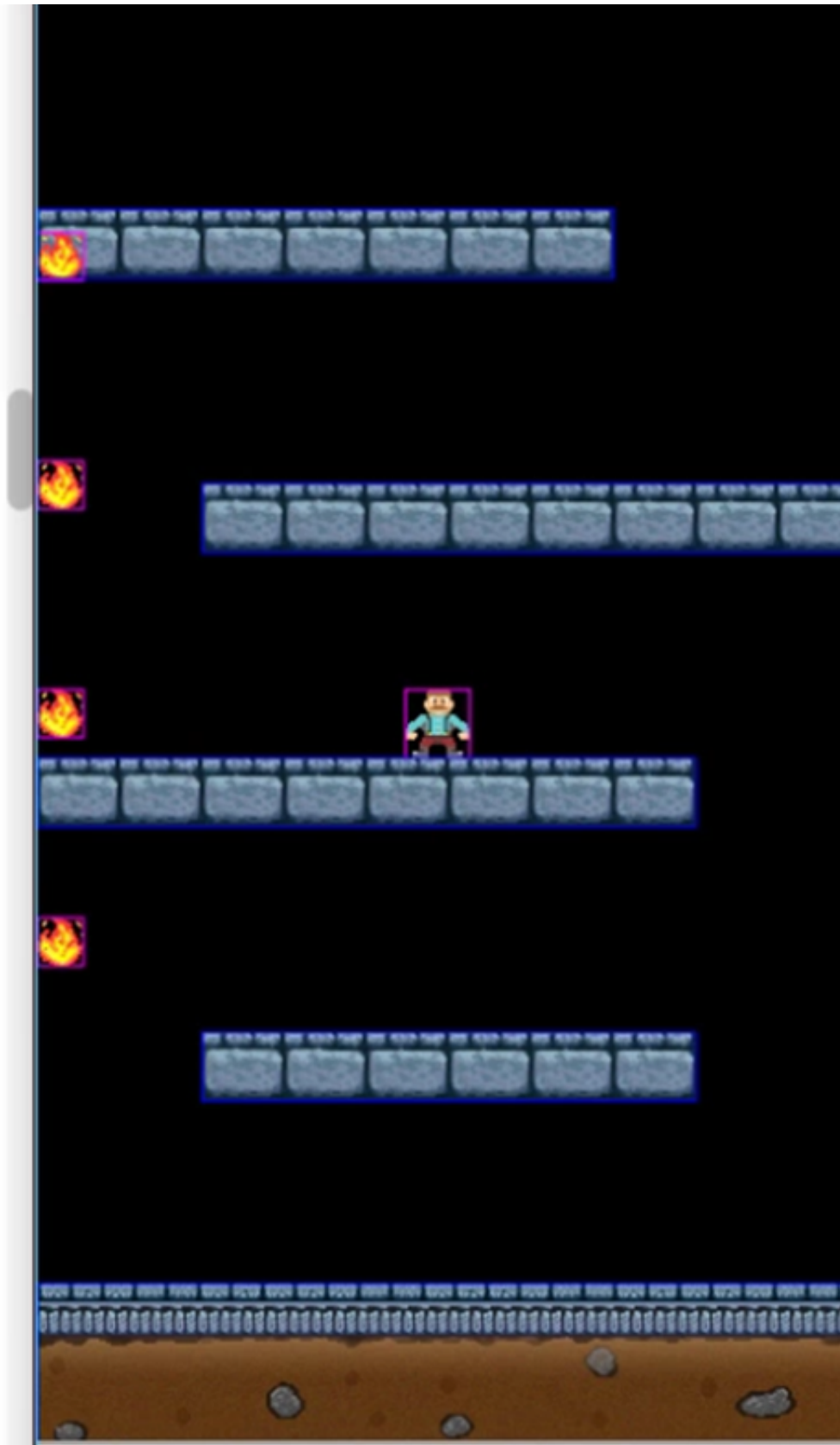
```
        frames: [0, 1]
    }},
    frameRate: 4,
    repeat: -1
});
```

Back down in our **setupLevel method**, we can now set up our **fires burning animation** to **play** right before we **add** the **fires object** to the game.

```
// play burning animation
newObj.ans.play('burning');
```

Fire Positioning

If you look at the game now, you should be able to see your fire sprites.



However, their placement isn't that great, and manually tweaking the numbers is once again tedious. What we can do to make the process easier, though, is to make the **fire elements draggable**. To start this process, in our **fires for loop** in **setupLevel**, we can **set** the **object** we add to be **interactive** and specify that it's **draggable**.



```
// this is for level creation
newObj.setInteractive();
this.input.setDraggable(newObj);
```

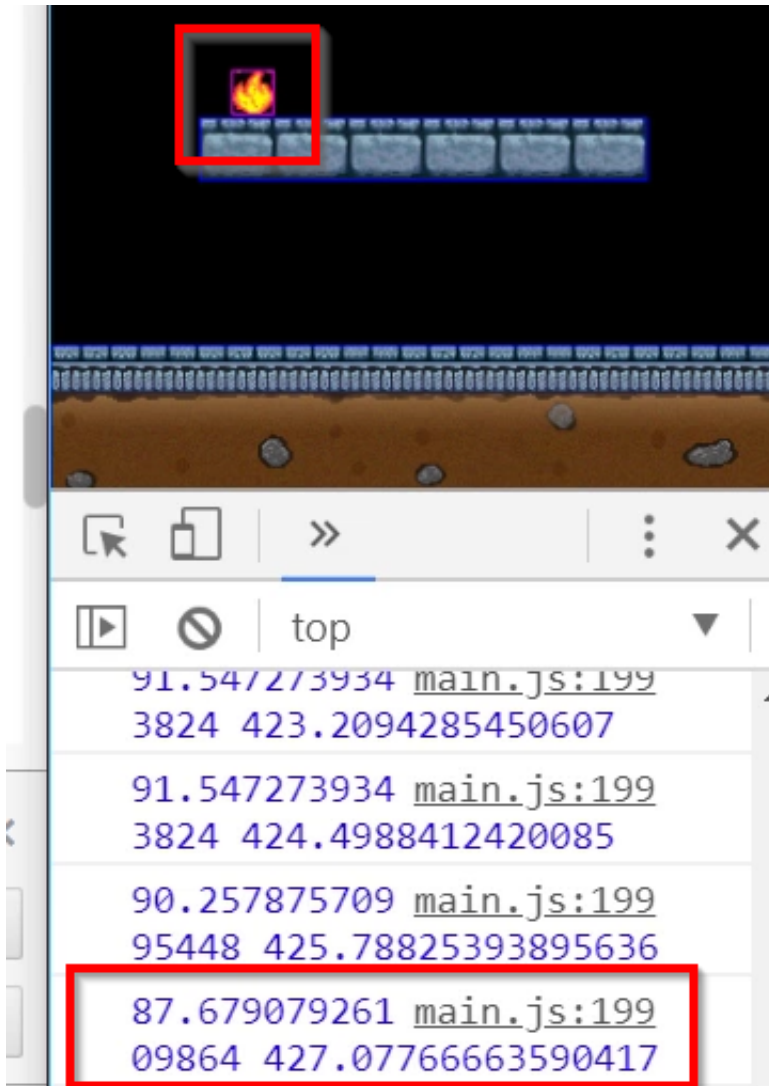
Outside of the **fires for loop**, we can then add a new **listener** that looks for the **dragging event**. With the attached **function**, we can take in our **pointer data** for the **drag** and the **gameObject**. Then, we can set the **gameObject's X and Y value** to match the position of the **pointer** while we're dragging. Since we're using this to determine our fire placement, we then want to **log** where we dragged the object so that we can get our positions in the **console**.

```
// for level creation
this.input.on('drag', function(pointer, gameObject, dragX, dragY){
    gameObject.x = dragX;
    gameObject.y = dragY;

    console.log(dragX, dragY);

});
```

After this, you can turn to your game and drag the fires around until you find positions you like. When you find a good placement, you can look to the **console** for the approximate coordinates to use.



With these, you can copy them into your **levelData.json** file. For us, our final coordinates wound up as follows:

```
"fires": [  
  {  
    "x": 12.9,  
    "y": 538.4  
  },  
  {  
    "x": 328,  
    "y": 188  
  },  
  {  
    "x": 68.3,  
    "y": 307  
  },  
  {  
    "x": 87.7,  
    "y": 427  
  }  
]
```



This is a useful method to design your level, but as you'll see later, we'll want to disable these parts when we're done creating the level.

Player and Goal Positioning

The last things we need to add for our **setupLevel method** are our **player** and **goal positioning**. From our **create method**, we're going to **cut** both where we **add** the **player sprite** and **physics** and where we **constrain** the **player** to the **world bounds**. Following, we will then **paste** it below our **code** for our level creation aspects we just did. Like with our previous sections, we'll want to plug in the **levelData** for the **X** and **Y** values.

Below the **player**, we can also **add** the **goal sprite** to the **world** as well and **add physics** to it. Again, we'll want to plug in our **levelData** for the **positioning**.

```
// player
this.player = this.add.sprite(this.levelData.player.x, this.levelData.player.y, 'player', 3);
this.physics.add.existing(this.player);

// constraint player to the game bounds
this.player.body.setCollideWorldBounds(true);

// goal
this.goal = this.add.sprite(this.levelData.goal.x, this.levelData.goal.y, 'goal');
this.physics.add.existing(this.goal);
```

Before we test this, we need to enable our **goal** to **collide** with the **platforms**. To do this, we can quickly scroll up to our **create function** and add a **collider** for the **goal** in our section for **collision detection**.

```
// collision detection
this.physics.add.collider(this.player, this.platforms);
this.physics.add.collider(this.goal, this.platforms);
```

At this point, we now have a fuller level.



In the next lesson, we will set up better **collision detection** so we can implement our **player** colliding with the fire.



In this lesson, we're going to set up the **collision detection** for overlapping objects and refactor our code some to be more efficient.

Groups vs. Physics Groups

At the moment, our game is setup to use regular **groups** to contain **objects** that have similar **physics**. While this is fine, whenever **Phaser** needs to detect **collisions**, it will check every object in that group. Thus, it is not the most performance efficient. However, if we instead use a **physics groups** for these elements, **Phaser** will internally use a tree structure for the calculations. Ultimately, this is faster for game performance.

Within the **setupLevel method** in **main.js**, we want to locate our **platforms group** setup and replace this **group** with a **Static Group**. This is a simple fix, but doing so will help our game. Further, we can still use all the same **API** elements we can usually do for **groups**, so our game will function as normal.

```
this.platforms = this.physics.add.staticGroup();
```

Further down, we also need to change our **fires group**. Since we want the option to be able to move the **fire** sprite, we will actually change this **group** to a **physics group** instead of a **static group**. However, due to this change, we need to move where we turn off the **gravity** and make the **fire immovable**. This is because our **physics group** settings override the settings we apply in the **for loop**. Fortunately, however, by passing in an **object** to the **physics group**, we can adjust the **gravity** and **immovable properties** similar to how we did before.

```
// create all the fire
this.fires = this.physics.add.group({
  allowGravity: false,
  immovable: true
});
for (let i = 0; i < this.levelData.fires.length; i++) {
  let curr = this.levelData.fires[i];

  let newObj = this.add.sprite(curr.x, curr.y, 'fire').setOrigin(0);

  // play burning animation
  newObj.anims.play('burning');

  // add to the group
  this.fires.add(newObj);

  // this is for level creation
  newObj.setInteractive();
  this.input.setDraggable(newObj);
}
```

Before we move on, we want to note it is possible to **create objects** in the **group** from the start, as seen with the line below. However, we are choosing not to do this for our game as **set Origin** doesn't function the same and moves the sprite away from the **body**. We could get around this by changing the **coordinates** or giving the **origin** an **offset**, but ultimately this is more work than we need to put in.

```
let newObj = this.fires.create(curr.x, curr.y, 'fire').setOrigin(0);
```



Overlap Collision Detection

The next feature we want for our game is the ability to restart the game when the player hits the fire or the goal. We will begin by working with our **collisions** in the **create function**. First, we're first going to condense our **colliders** to a single line and use an **array** to mark what is colliding with the **platforms**.

```
// collision detection  
this.physics.add.collider([this.player, this.goal], this.platforms);
```

Next, we will set up our **overlap**. Unlike a **collider**, which physically stops an object from moving



through it, an **overlap physics object** allows **objects** to pass through but is able to detect when two objects have touched. Like the **collider**, we need to pass in the **two objects/groups** we want to detect the **overlap** for. However, for **overlap**, we also need to pass in three additional parameters. First, we need a **callback function** that is run when the **overlap** occurs, which we will call in our code and create the **function** for it in a moment. Second, we need an optional **parameter** that checks if we even want to run our **callback function**. We don't really need this, so we'll be passing in **null**. Third is our **context**, which in this case is simply **this**.

```
// overlap checks
this.physics.add.overlap(this.player, [this.fires, this.goal], this.restartGame, null
, this);
```

(**Note:** Only **physics groups** can be passed in as an **array** for **collisions**. Regular groups are not valid).

Next, under our **setupLevel method**, we're going to add our **restartGame function**. First within the **method**, we want to target the **main camera** and **fade** out our **scene** for half a second (**500 ms**). After which, we want to set up a **listener** that will **listen** for the **camera's fade out** to be **completed**. In this **listener**, we need to both pass in a **callback function** that actually **restarts** our **scene** and the **context** (which again, is **this**).

While we won't be using them, the code below also shows optional **parameters** you can use if you'd like to tweak parts of the code. In this case, our **restartGame function** will know the **sprite** that triggered the **overlap** (**sourceSprite**) and the other **sprite** it overlapped with (the **targetsprite**). Meanwhile, our **callback function** for our **camera's fade out event** will know about the **camera** and **effect**, so it is able to use those for whatever you might want in the **function**.

```
// restart game (game over + you won!)
gameScene.restartGame = function(sourceSprite, targetSprite){
  // fade out
  this.cameras.main.fade(500);

  // when fade out completes, restart scene
  this.cameras.main.on('camerafadeoutcomplete', function(camera, effect){
    // restart the scene
    this.scene.restart();
  }, this);
};
```

After this, we should be able to run around and hit the **fires** or the **goal** and see the scene restart!

elDa



Animations and Cameras

Moving on, we need to make a note about our **animations**. Since we **create** these in our **create method**, they are global and available in all other scenes. However, when we **restart** the scene as we did, **Phaser** warns us that the **key** for that **animation** already exists. This is because when we **restart** the **scene**, **Phaser** tries to create the animation anew even though it already exists. There are a few ways we can deal with this though. First, we could create another scene whose sole job is to **preload** our **assets** and is only ever called once in the game. Second, and the technique we will use, is to simply check **if** the **animation** already exists with an **if statement**. **If** it doesn't exist, we'll create the **animation** as we did before.

```
if(!this.anims.get('walking')) {
  // walking animation
  this.anims.create({
    key: 'walking',
    frames: this.anims.generateFrameNames('player', {
      frames: [0, 1, 2]
    }),
    frameRate: 12,
    yoyo: true,
    repeat: -1
  });
}

if(!this.anims.get('burning')) {
  // fire animation
  this.anims.create({
    key: 'burning',
    frames: this.anims.generateFrameNames('fire', {
      frames: [0, 1]
    }),
    frameRate: 4,
    repeat: -1
  });
}
```



Before we close out, there is one last thing we want to do in this lesson: make the **camera** follow the **player**. Within the **create function**, we can set this up under where we declare the **world bounds**. First, we need to define the **bounds** for the **camera** itself by passing in the **origin** and the **width** and **height** of the **bounds**. After doing this, we can use **startFollow** and tell it to **follow** our **player**. However, we need to move **setupLevel** to appear before our **world bounds** for now, as without this our **player** isn't defined yet for the **camera** to know who to follow.

```
// add all level elements
this.setupLevel();

// world bounds
this.physics.world.bounds.width = 360;
this.physics.world.bounds.height = 700;

// camera bounds
this.cameras.main.setBounds(0, 0, 360, 700);
this.cameras.main.startFollow(this.player);
```

In the next lesson, we will set up our **barrel generation**. If you'd like to learn more about **overlap physics**, please be sure to consult the following **documentation**:
https://photonstorm.github.io/phaser3-docs/Phaser.Physics.Arcade.ArcadePhysics.html#overlap_anchor



In this lesson, we're going to make the gorilla throw barrels.

JSON File Additions and World Bounds

We're first going to begin in our **levelData.json** file. At the top of the **JSON object**, we're going to add two more **keys**. The first **key** will be for our **world**, and we'll use this to define our **width** and **height** for the **world bounds**. In so doing, this can be later included in our **level setup** so we are using less hard-coded numbers. Second, we're going to add a **key** for a **spawner**. This **spawner** is what will generate the **barrels**, and we want to know how frequently they spawn, their speed, and how long the barrels will exist in the game.

```
"world": {  
  "width": 360,  
  "height": 700  
},  
"spawner": {  
  "interval": 1000,  
  "speed": 100,  
  "lifespan": 3000  
},  
//More code
```

Next, back in **main.js**, we're going to turn to our **create function** and **cut** both our **world bounds** creation and our **camera follow** setup from that **function**. Within the **setupLevel function**, we will **paste** our **world bounds creation** under where we **load** in our **levelData.json file**. Since the goal is to use non-hardcoded numbers, we can apply the **levelData world data** we just set up to define the values.

```
// world bounds  
this.physics.world.bounds.width = this.levelData.world.width;  
this.physics.world.bounds.height = this.levelData.world.height;
```

Then, further down under where we **constrain** the **player** to the **world bounds**, we can **paste** our **camera bounds** and **follow setup**. Like before, we want to plug in our **world levelData** where appropriate.

```
// camera bounds  
this.cameras.main.setBounds(0, 0, this.levelData.world.width, this.levelData.world.height);  
this.cameras.main.startFollow(this.player);
```

Our game should function as before, but now we can also alter the size of the game from our **JSON** file. Once again, this can be convenient for updating levels quickly.

Setting up the Spawner

Now that our **world bounds** are more dynamic, we can deal with our **spawner** which will be in the same position as our **goal**. Under our **restartGame method**, we're going to create a new **method** called **setupSpawner**. To begin in this **method**, we're going to create a **barrels**



physics group to store our **barrels**. We want our **barrels** to have a little bit of **bounce**, so we're going to adjust the **X** and **Y axes** properties for the **bounce property** on them. Further, we don't want the **barrel** to fall off the edge, so we're going to make sure it also **collides** with the **world bounds**.

```
// generation of barrels
gameScene.setupSpawner = function(){
  // barrel group
  this.barrels = this.physics.add.group({
    bounceY: 0.1,
    bounceX: 1,
    collideWorldBounds: true
  });
};
```

Next, we can set up the actual elements that **spawn** our **barrels**. In this case, we need to run an **event** that recurs at a set interval. To do this, we can set up a **spawningEvent** and call **time.addEvent** to it. In this **addEvent function**, we can define all the **properties** of our **event**. In this case, we will have a **delay**, which uses the **spawner interval** from **levelData**, a **loop** set to **true** so the event happens over and over again, and the **callback information** on what the **event** actually does.

```
// generation of barrels
gameScene.setupSpawner = function(){
  // barrel group
  this.barrels = this.physics.add.group({
    bounceY: 0.1,
    bounceX: 1,
    collideWorldBounds: true
  });

  // spawn barrels
  let spawningEvent = this.time.addEvent({
    delay: this.levelData.spawner.interval,
    loop: true,
    callbackScope: this,
    callback: function(){

    }
  });
};
```

With the **callback function**, there are three primary things we need to do. First, we need to **create** our **barrel object** in the same way we've done with other sprites using the **goal's position** as its **starting** coordinates. After that, we need to set the **barrel's properties**. In this case, we want to make sure to set the **barrel's X velocity** to be what we set as the **spawner's speed**. This will get our barrels moving. Last, we need to set up the **barrel's lifespan**. However, our **lifespan** itself need to be a non-repeating **event**. It will be similar to the **event** we created originally, only using the **spawner's lifespan property** as the **delay** and using the **callback function** to **destroy** the **barrel object**.



```
// generation of barrels
gameScene.setupSpawner = function(){
  // barrel group
  this.barrels = this.physics.add.group({
    bounceY: 0.1,
    bounceX: 1,
    collideWorldBounds: true
  });

  // spawn barrels
  let spawningEvent = this.time.addEvent({
    delay: this.levelData.spawner.interval,
    loop: true,
    callbackScope: this,
    callback: function(){
      // create a barrel
      let barrel = this.barrels.create(this.goal.x, this.goal.y, 'barrel');

      // set properties
      barrel.setVelocityX(this.levelData.spawner.speed);

      // lifespan
      this.time.addEvent({
        delay: this.levelData.spawner.lifespan,
        repeat: 0,
        callbackScope: this,
        callback: function(){
          barrel.destroy();
        }
      });
    }
  });
};
```

Finishing the Barrels

Finally, back in **create**, we can **call** our **setupSpawner method** under where we call **setupLevel**. Further, since we have a new **barrels group**, we want to be sure to add this to both our **collider** and **overlap detections** so we can actually interact with the **barrels**.

```
// add all level elements
this.setupLevel();

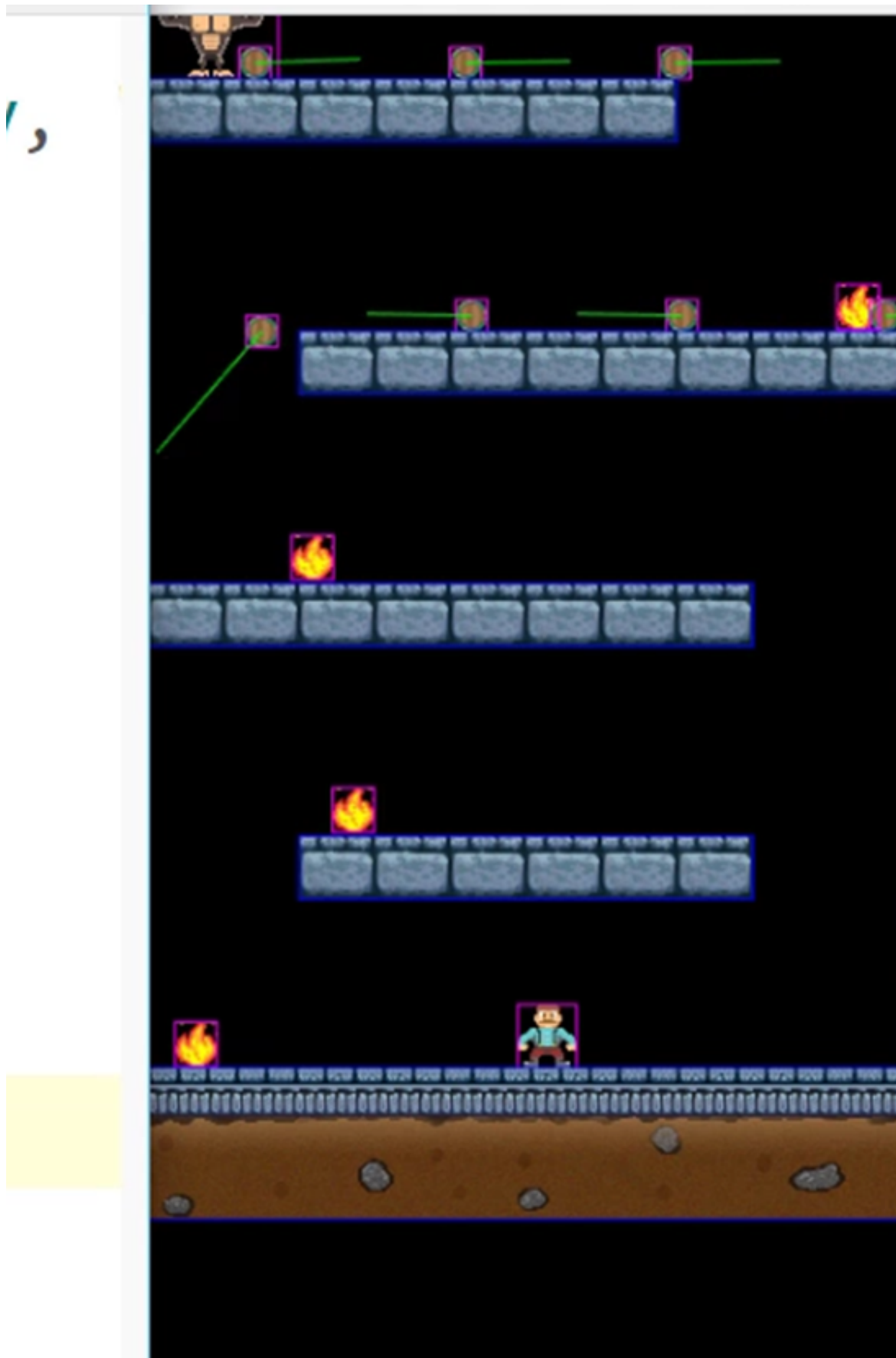
// initiate barrel spawner
this.setupSpawner();

// collision detection
this.physics.add.collider([this.player, this.goal, this.barrels], this.platforms);

// overlap checks
this.physics.add.overlap(this.player, [this.fires, this.goal, this.barrels], this.restartGame, null, this);
```



After this, you should be able to see your **barrels** spawn, roll, and eventually disappear.





In the next and final lesson before we wrap up, we're going to make our **barrel** spawning more efficient by **recycling** already created **barrels**.

In this final lesson of the project, we're going to implement **object pooling** for our barrels.

What is an Object Pool?

First, let's talk about what an **object pool** even is. An **object pool** is a collection of objects from which we "create" objects in the game. Instead of making new objects every time we need them, we instead create a **pool** that we can pull from. This **pool** will contain active and inactive **objects**, and any time we need a new **object**, we can find an inactive **object** in the **pool** and re-purpose it.

There are a few advantages to using objects in this way. To begin, **creating** and **destroying objects** uses a lot of **CPU** power compared to re-purposing them. Further, **Javascript** has what's known as a **garbage collector**, which is not something as a developer we have control over. Due to this, even when we **destroy** the **objects**, the **garbage collector** may leave these **objects** in the game for an extended period before collecting them. Overall, these aspects will slow down your game.

Barrel Object Pool

Fortunately, **Phaser 3** comes with a quick, built-in way to set up **object pooling**, so we can utilize this method to create an **object pool** for our **barrels**. Within **main.js** in the **setupSpawner method**, we want to locate where we **create** the **barrel** in our **spawningEvent**. In order to create a **pool**, we just need to use the **get function** instead of **create**. In so doing, **Phaser** will automatically pull **inactive objects** from the **pool**. However, **if** there aren't any **inactive** objects, **Phaser** will automatically **create** an **object** for you.

```
// create a barrel
let barrel = this.barrels.get(this.goal.x, this.goal.y, 'barrel');
```

Since we're pulling from a **pool**, instead of **destroying** the **barrels** at the end of their **lifespan**, we want to simply make them **inactive** when their lifespan ends. To do this, we can use a **method** called **killAndHide** on them. However, this will only target the **sprite**. To target the **physics body**, we also need to **disable** that by setting its **enable property** to **false**.

```
// lifespan
this.time.addEvent({
  delay: this.levelData.spawner.lifespan,
  repeat: 0,
  callbackScope: this,
  callback: function(){
    this.barrels.killAndHide(barrel);
    barrel.body.enable = false;
  }
});
```

Next, since we're now pulling from the **pool**, we also need a way to **reactivate** the **barrels** when they're called upon. Under where we **create** the **barrel**, we can do this by **setting** the **object** to be both **Active** and **Visible**. In addition, we'll also want to make sure to **re-enable** the **body physics**.

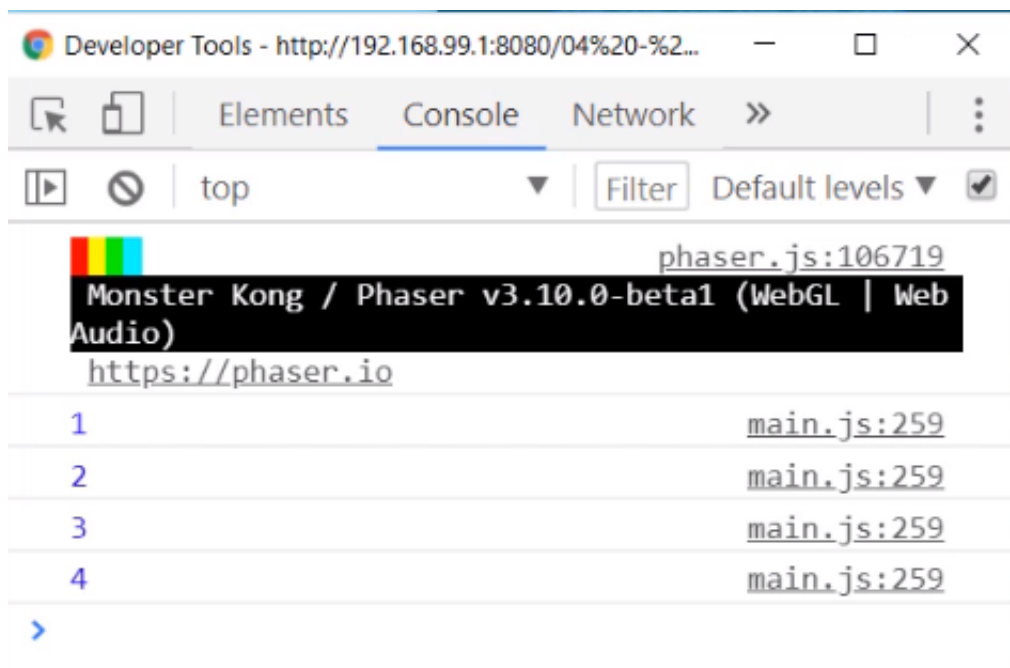
```
// create a barrel
let barrel = this.barrels.get(this.goal.x, this.goal.y, 'barrel');
```

```
// reactivate
barrel.setActive(true);
barrel.setVisible(true);
barrel.body.enable = true;

// set properties
barrel.setVelocityX(this.levelData.spawner.speed);
```

Note that if you'd like to be able to prove the **object pooling** is working, you can add a **console log** before the **lifespan section** to **print** all the **children** that are part of the **barrels group**. Upon doing so, you'll be able to see the game create only as many children as are needed.

```
console.log(this.barrels.getChildren().length);
```

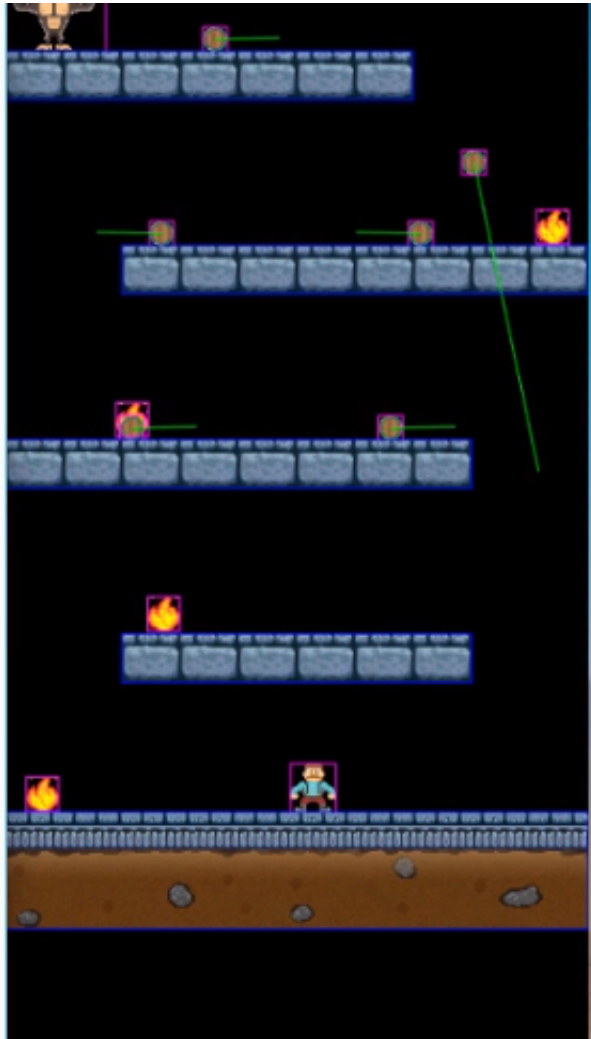


Before we declare the project done, let's fix our spawner parameters. In the **levelData.json file**, we're going to tweak our **spawner values** so the **barrels** actually present a challenge and can make it all the way to the bottom. You are free to change them as you see fit, but you can see our choices via the code below.

```
"spawner": {
  "interval": 3000,
  "speed": 80,
  "lifespan": 30000
},
```



a)



Congratulations! You've complete the main project portion of **Zenva's Craft a Mario-Style Platformer in Phaser 3** course! If you need any supplemental information for the topics covered in this course, be sure consult the **Phaser 3 documentation**:
<https://photonstorm.github.io/phaser3-docs/index.html>



Right now, if you press the left key while the player is moving right it will immediately start moving left, even though the same does not happen if you press the right key while the player is moving right.

We can fix this by changing the update loop to be as below:

```
gameScene.update = function() {
  // are we on the ground?
  let onGround = this.player.body.blocked.down || this.player.body.touching.down;

  // movement to the left
  if (this.cursors.left.isDown && !this.cursors.right.isDown) {
    this.player.body.setVelocityX(-this.playerSpeed);

    this.player.flipX = false;

    // play animation if none is playing
    if (onGround && !this.player.anims.isPlaying)
      this.player.anims.play('walking');
  }

  // movement to the right
  else if (this.cursors.right.isDown && !this.cursors.left.isDown) {
    this.player.body.setVelocityX(this.playerSpeed);

    this.player.flipX = true;

    // play animation if none is playing
    if (onGround && !this.player.anims.isPlaying)
      this.player.anims.play('walking');
  } else if (!this.cursors.left.isDown && !this.cursors.right.isDown) {
    // make the player stop
    this.player.body.setVelocityX(0);

    // stop walking animation
    this.player.anims.stop('walking');

    // set default frame
    if (onGround)
      this.player.setFrame(3);
  }

  // handle jumping
  if (onGround && (this.cursors.space.isDown || this.cursors.up.isDown)) {
    // give the player a velocity in Y
    this.player.body.setVelocityY(this.jumpSpeed);

    // stop the walking animation
    this.player.anims.stop('walking');

    // change frame
    this.player.setFrame(2);
  }
};
```



With this code, the player will change direction only if it stops moving in the previous direction.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Starting

index.html

Found in Project/00 - Starting

This HTML file sets up a basic Phaser game environment. It imports Phaser's JavaScript library and a file named `main.js` containing the logic for the intended game. The CSS styles apply minimalistic settings to the page's body and the game canvas elements, intended for full width display with no margin.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <script src="js/phaser.js"></script>
  <style>
    body {
      margin: 0;
    }

    canvas {
      width: 100%;
    }
  </style>
</head>
<body>
  <script src="js/main.js"></script>
</body>
</html>
```

main.js

Found in Project/00 - Starting/js

This JavaScript code sets up a Phaser game for a platformer game named 'Monster Kong'. A new game scene is created, importing the assets necessary for gameplay such as different types of game elements and sprite sheets for the player and fire animations. Then it constructs a new `Phaser.Game` with defined configurations such as type, dimensions, and the game scene.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// some parameters for our scene
gameScene.init = function() {};

// load asset files for our game
```

```
gameScene.preload = function() {

    // load images
    this.load.image('ground', 'assets/images/ground.png');
    this.load.image('platform', 'assets/images/platform.png');
    this.load.image('block', 'assets/images/block.png');
    this.load.image('goal', 'assets/images/gorilla3.png');
    this.load.image('barrel', 'assets/images/barrel.png');

    // load spritesheets
    this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
        frameWidth: 28,
        frameHeight: 30,
        margin: 1,
        spacing: 1
    });

    this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
        frameWidth: 20,
        frameHeight: 21,
        margin: 1,
        spacing: 1
    });
};

// executed once, after assets were loaded
gameScene.create = function() {};

// our game's configuration
let config = {
    type: Phaser.AUTO,
    width: 360,
    height: 640,
    scene: gameScene,
    title: 'Monster Kong',
    pixelArt: false
};

// create the game, and pass it the configuration
let game = new Phaser.Game(config);
```

Arcade Basics

main.js

Found in Project/01 - Arcade Basics/js

The code establishes a basic game setup using the Phaser.js library. It begins by creating a new game scene, then loads image assets and sprite sheets with different parameters. Finally, sprites are added into the game's physics system and the game is initialized with configuration properties such as canvas size, arcade physics settings, and scene designations.

```
// create a new scene
```

```
let gameScene = new Phaser.Scene('Game');

// some parameters for our scene
gameScene.init = function() {};

// load asset files for our game
gameScene.preload = function() {

    // load images
    this.load.image('ground', 'assets/images/ground.png');
    this.load.image('platform', 'assets/images/platform.png');
    this.load.image('block', 'assets/images/block.png');
    this.load.image('goal', 'assets/images/gorilla3.png');
    this.load.image('barrel', 'assets/images/barrel.png');

    // load spritesheets
    this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
        frameWidth: 28,
        frameHeight: 30,
        margin: 1,
        spacing: 1
    });

    this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
        frameWidth: 20,
        frameHeight: 21,
        margin: 1,
        spacing: 1
    });
};

// executed once, after assets were loaded
gameScene.create = function() {

    // 1) adding existing sprites to the physics system
    // sprite creation
    let ground = this.add.sprite(180, 400, 'ground');

    // add sprite to the physics system
    this.physics.add.existing(ground);

    // 2) creating and adding sprites to the physics system
    let ground2 = this.physics.add.sprite(180, 200, 'ground');

    console.log(ground2);
};

// our game's configuration
let config = {
    type: Phaser.AUTO,
    width: 360,
    height: 640,
    scene: gameScene,
    title: 'Monster Kong',
    pixelArt: false,
```

```
physics: {
  default: 'arcade',
  arcade: {
    gravity: {y: 1000},
    debug: true
  }
}
};

// create the game, and pass it the configuration
let game = new Phaser.Game(config);
```

Static and Dynamic Bodies

main.js

Found in Project/02 - Static and Dynamic Bodies/js

The code constructs a Phaser game scene where it first preloads image and sprite assets. Then, it creates a playable character, a ground platform and a separate movable platform using those assets. Lastly, it sets up the game configuration and initializes the game with it.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// some parameters for our scene
gameScene.init = function() {};

// load asset files for our game
gameScene.preload = function() {

  // load images
  this.load.image('ground', 'assets/images/ground.png');
  this.load.image('platform', 'assets/images/platform.png');
  this.load.image('block', 'assets/images/block.png');
  this.load.image('goal', 'assets/images/gorilla3.png');
  this.load.image('barrel', 'assets/images/barrel.png');

  // load spritesheets
  this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
    frameWidth: 28,
    frameHeight: 30,
    margin: 1,
    spacing: 1
  });

  this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
    frameWidth: 20,
    frameHeight: 21,
    margin: 1,
    spacing: 1
  });
};
```

```
// executed once, after assets were loaded
gameScene.create = function() {

    this.platforms = this.add.group();

    // ground
    let ground = this.add.sprite(180, 604, 'ground');
    this.physics.add.existing(ground, true);
    this.platforms.add(ground);

    // platform
    let platform = this.add.tileSprite(180, 500, 4 * 36, 1 * 30, 'block');
    this.physics.add.existing(platform, true);
    this.platforms.add(platform);

    // player
    this.player = this.add.sprite(180, 400, 'player', 3);
    this.physics.add.existing(this.player);

    // disable gravity
    //ground.body.allowGravity = false;

    // make it immovable
    //ground.body.immovable = true;

    // 2) creating and adding sprites to the physics system
    //let ground2 = this.physics.add.sprite(180, 200, 'ground');

    // collision detection
    //this.physics.add.collider(ground, ground2);
    this.physics.add.collider(this.player, this.platforms);
};

// our game's configuration
let config = {
    type: Phaser.AUTO,
    width: 360,
    height: 640,
    scene: gameScene,
    title: 'Monster Kong',
    pixelArt: false,
    physics: {
        default: 'arcade',
        arcade: {
            gravity: {y: 1000},
            debug: true
        }
    }
};

// create the game, and pass it the configuration
let game = new Phaser.Game(config);
```

Player Walking

main.js

Found in Project/03 - Player Walking/js

The code constructs a Phaser game with player movement and animation. It starts by setting speed parameters for the player, then it loads game assets. After creating the game scene with platforms and a player sprite, it sets up walking animation for the player and enables keyboard controls. The update function handles the player's movement and animation based on keyboard inputs.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// some parameters for our scene
gameScene.init = function() {

    // player parameters
    this.playerSpeed = 150;
    this.jumpSpeed = -600;
};

// load asset files for our game
gameScene.preload = function() {

    // load images
    this.load.image('ground', 'assets/images/ground.png');
    this.load.image('platform', 'assets/images/platform.png');
    this.load.image('block', 'assets/images/block.png');
    this.load.image('goal', 'assets/images/gorilla3.png');
    this.load.image('barrel', 'assets/images/barrel.png');

    // load spritesheets
    this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
        frameWidth: 28,
        frameHeight: 30,
        margin: 1,
        spacing: 1
    });

    this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
        frameWidth: 20,
        frameHeight: 21,
        margin: 1,
        spacing: 1
    });
};

// executed once, after assets were loaded
gameScene.create = function() {

    this.platforms = this.add.group();

    // ground
    let ground = this.add.sprite(180, 604, 'ground');
```



```
this.physics.add.existing(ground, true);
this.platforms.add(ground);

// platform
let platform = this.add.tileSprite(180, 500, 4 * 36, 1 * 30, 'block');
this.physics.add.existing(platform, true);
this.platforms.add(platform);

// player
this.player = this.add.sprite(180, 400, 'player', 3);
this.physics.add.existing(this.player);

// walking animation
this.anims.create({
  key: 'walking',
  frames: this.anims.generateFrameNames('player', {
    frames: [0, 1, 2]
  }),
  frameRate: 12,
  yoyo: true,
  repeat: -1
});

// collision detection
this.physics.add.collider(this.player, this.platforms);

// enable cursor keys
this.cursors = this.input.keyboard.createCursorKeys();
};

// executed on every frame
gameScene.update = function()
{
  // movement to the left
  if(this.cursors.left.isDown) {
    this.player.body.setVelocityX(-this.playerSpeed);

    this.player.flipX = false;

    // play animation if none is playing
    if(!this.player.anims.isPlaying)
      this.player.anims.play('walking');
  }

  // movement to the right
  else if(this.cursors.right.isDown) {
    this.player.body.setVelocityX(this.playerSpeed);

    this.player.flipX = true;

    // play animation if none is playing
    if(!this.player.anims.isPlaying)
      this.player.anims.play('walking');
  }
  else {
```



```
// make the player stop
this.player.body.setVelocityX(0);

// stop walking animation
this.player.anims.stop('walking');

// set default frame
this.player.setFrame(3);
}
};

// our game's configuration
let config = {
  type: Phaser.AUTO,
  width: 360,
  height: 640,
  scene: gameScene,
  title: 'Monster Kong',
  pixelArt: false,
  physics: {
    default: 'arcade',
    arcade: {
      gravity: {y: 1000},
      debug: true
    }
  }
};

// create the game, and pass it the configuration
let game = new Phaser.Game(config);
```

Player Jumping

main.js

Found in Project/04 - Player Jumping/js

The code establishes a Phaser game scene with player controls for movements and jumping. After defining game parameters and loading assets, it creates the game scene, setting game world bounds and ensuring the player can't leave them. Then it enables keyboard-based player controls for moving left, right or jumping. The game also features frame-specific animations for different player actions.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// some parameters for our scene
gameScene.init = function() {

  // player parameters
  this.playerSpeed = 150;
  this.jumpSpeed = -600;
};
```



```
// load asset files for our game
gameScene.preload = function() {

    // load images
    this.load.image('ground', 'assets/images/ground.png');
    this.load.image('platform', 'assets/images/platform.png');
    this.load.image('block', 'assets/images/block.png');
    this.load.image('goal', 'assets/images/gorilla3.png');
    this.load.image('barrel', 'assets/images/barrel.png');

    // load spritesheets
    this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
        frameWidth: 28,
        frameHeight: 30,
        margin: 1,
        spacing: 1
    });

    this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
        frameWidth: 20,
        frameHeight: 21,
        margin: 1,
        spacing: 1
    });
};

// executed once, after assets were loaded
gameScene.create = function() {

    // world bounds
    this.physics.world.bounds.width = 360;
    this.physics.world.bounds.height = 700;

    this.platforms = this.add.group();

    // ground
    let ground = this.add.sprite(180, 604, 'ground');
    this.physics.add.existing(ground, true);
    this.platforms.add(ground);

    // platform
    let platform = this.add.tileSprite(180, 500, 4 * 36, 1 * 30, 'block');
    this.physics.add.existing(platform, true);
    this.platforms.add(platform);

    // player
    this.player = this.add.sprite(180, 400, 'player', 3);
    this.physics.add.existing(this.player);

    // constrain player to the game bounds
    this.player.body.setCollideWorldBounds(true);

    // walking animation
    this.anims.create({
```



```
    key: 'walking',
    frames: this.anims.generateFrameNames('player', {
      frames: [0, 1, 2]
    }),
    frameRate: 12,
    yoyo: true,
    repeat: -1
  });

  // collision detection
  this.physics.add.collider(this.player, this.platforms);

  // enable cursor keys
  this.cursors = this.input.keyboard.createCursorKeys();
};

// executed on every frame
gameScene.update = function()
{
  // are we on the ground?
  let onGround = this.player.body.blocked.down ||
this.player.body.touching.down;

  // movement to the left
  if(this.cursors.left.isDown) {
    this.player.body.setVelocityX(-this.playerSpeed);

    this.player.flipX = false;

    // play animation if none is playing
    if(onGround && !this.player.anims.isPlaying)
      this.player.anims.play('walking');
  }

  // movement to the right
  else if(this.cursors.right.isDown) {
    this.player.body.setVelocityX(this.playerSpeed);

    this.player.flipX = true;

    // play animation if none is playing
    if(onGround && !this.player.anims.isPlaying)
      this.player.anims.play('walking');
  }
  else {
    // make the player stop
    this.player.body.setVelocityX(0);

    // stop walking animation
    this.player.anims.stop('walking');

    // set default frame
    if(onGround)
      this.player.setFrame(3);
  }
}
```

```
// handle jumping
if(onGround && (this.cursors.space.isDown || this.cursors.up.isDown)) {
  // give the player a velocity in Y
  this.player.body.setVelocityY(this.jumpSpeed);

  // stop the walking animation
  this.player.anims.stop('walking');

  // change frame
  this.player.setFrame(2);
}
};

// our game's configuration
let config = {
  type: Phaser.AUTO,
  width: 360,
  height: 640,
  scene: gameScene,
  title: 'Monster Kong',
  pixelArt: false,
  physics: {
    default: 'arcade',
    arcade: {
      gravity: {y: 1000},
      debug: true
    }
  }
};

// create the game, and pass it the configuration
let game = new Phaser.Game(config);
```

Level Data and JSON

main.js

Found in Project/05 - Level Data and JSON/js

The code establishes a Phaser game using a JSON file to set up level platforms. After loading game assets and configuring player parameters, the level is automatically created using data from the levelData.json file. Player movement controls, physics, and collision detection are then set up. The game also detects and logs pointer down events on the game canvas.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// some parameters for our scene
gameScene.init = function() {

  // player parameters
  this.playerSpeed = 150;
```

```
this.jumpSpeed = -600;
};

// load asset files for our game
gameScene.preload = function() {

    // load images
    this.load.image('ground', 'assets/images/ground.png');
    this.load.image('platform', 'assets/images/platform.png');
    this.load.image('block', 'assets/images/block.png');
    this.load.image('goal', 'assets/images/gorilla3.png');
    this.load.image('barrel', 'assets/images/barrel.png');

    // load spritesheets
    this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
        frameWidth: 28,
        frameHeight: 30,
        margin: 1,
        spacing: 1
    });

    this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
        frameWidth: 20,
        frameHeight: 21,
        margin: 1,
        spacing: 1
    });

    this.load.json('levelData', 'assets/json/levelData.json');
};

// executed once, after assets were loaded
gameScene.create = function() {

    // world bounds
    this.physics.world.bounds.width = 360;
    this.physics.world.bounds.height = 700;

    // add all level elements
    this.setupLevel();

    // player
    this.player = this.add.sprite(175, 280, 'player', 3);
    this.physics.add.existing(this.player);

    // constraint player to the game bounds
    this.player.body.setCollideWorldBounds(true);

    // walking animation
    this.anims.create({
        key: 'walking',
        frames: this.anims.generateFrameNames('player', {
            frames: [0, 1, 2]
        }),
        frameRate: 12,
```

```
        yoyo: true,
        repeat: -1
    });

    // collision detection
    this.physics.add.collider(this.player, this.platforms);

    // enable cursor keys
    this.cursors = this.input.keyboard.createCursorKeys();

    this.input.on('pointerdown', function(pointer) {
        console.log(pointer.x, pointer.y);
    });
};

// executed on every frame
gameScene.update = function() {
    // are we on the ground?
    let onGround = this.player.body.blocked.down ||
        this.player.body.touching.down;

    // movement to the left
    if (this.cursors.left.isDown) {
        this.player.body.setVelocityX(-this.playerSpeed);

        this.player.flipX = false;

        // play animation if none is playing
        if (onGround && !this.player.anims.isPlaying)
            this.player.anims.play('walking');
    }

    // movement to the right
    else if (this.cursors.right.isDown) {
        this.player.body.setVelocityX(this.playerSpeed);

        this.player.flipX = true;

        // play animation if none is playing
        if (onGround && !this.player.anims.isPlaying)
            this.player.anims.play('walking');
    } else {
        // make the player stop
        this.player.body.setVelocityX(0);

        // stop walking animation
        this.player.anims.stop('walking');

        // set default frame
        if (onGround)
            this.player.setFrame(3);
    }

    // handle jumping
    if (onGround && (this.cursors.space.isDown || this.cursors.up.isDown)) {
```

```
// give the player a velocity in Y
this.player.body.setVelocityY(this.jumpSpeed);

// stop the walking animation
this.player.anims.stop('walking');

// change frame
this.player.setFrame(2);
}
};

// sets up all the elements in the level
gameScene.setupLevel = function() {
  this.platforms = this.add.group();

  // parse json data
  this.levelData = this.cache.json.get('levelData');

  // create all the platforms
  for (let i = 0; i < this.levelData.platforms.length; i++) {
    let curr = this.levelData.platforms[i];

    let newObj;

    // create object
    if(curr.numTiles == 1) {
      // create sprite
      newObj = this.add.sprite(curr.x, curr.y, curr.key).setOrigin(0);
    }
    else {
      // create tilesprite
      let width = this.textures.get(curr.key).get(0).width;
      let height = this.textures.get(curr.key).get(0).height;
      newObj = this.add.tileSprite(curr.x, curr.y, curr.numTiles * width, height, curr.key).setOrigin(0);
    }

    // enable physics
    this.physics.add.existing(newObj, true);

    // add to the group
    this.platforms.add(newObj);
  }
};

// our game's configuration
let config = {
  type: Phaser.AUTO,
  width: 360,
  height: 640,
  scene: gameScene,
  title: 'Monster Kong',
  pixelArt: false,
  physics: {
```

```
    default: 'arcade',
    arcade: {
      gravity: {
        y: 1000
      },
      debug: true
    }
  }
};

// create the game, and pass it the configuration
let game = new Phaser.Game(config);
```

Fire and Positions

main.js

Found in Project/06 - Fire and Positions/js

The code establishes a Phaser game with a level setup using JSON data. The game scene includes character controlling, walking and burning animations, and interactive setting for level designing. Additionally, it has a physics engine to detect collisions between the player, platforms, fires, and a goal. It also logs the position of pointer-down events.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// some parameters for our scene
gameScene.init = function() {

  // player parameters
  this.playerSpeed = 150;
  this.jumpSpeed = -600;
};

// load asset files for our game
gameScene.preload = function() {

  // load images
  this.load.image('ground', 'assets/images/ground.png');
  this.load.image('platform', 'assets/images/platform.png');
  this.load.image('block', 'assets/images/block.png');
  this.load.image('goal', 'assets/images/gorilla3.png');
  this.load.image('barrel', 'assets/images/barrel.png');

  // load spritesheets
  this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
    frameWidth: 28,
    frameHeight: 30,
    margin: 1,
    spacing: 1
  });
};
```



```
this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
  frameWidth: 20,
  frameHeight: 21,
  margin: 1,
  spacing: 1
});

this.load.json('levelData', 'assets/json/levelData.json');
};

// executed once, after assets were loaded
gameScene.create = function() {
  // walking animation
  this.anims.create({
    key: 'walking',
    frames: this.anims.generateFrameNames('player', {
      frames: [0, 1, 2]
    }),
    frameRate: 12,
    yoyo: true,
    repeat: -1
  });

  // fire animation
  this.anims.create({
    key: 'burning',
    frames: this.anims.generateFrameNames('fire', {
      frames: [0, 1]
    }),
    frameRate: 4,
    repeat: -1
  });

  // world bounds
  this.physics.world.bounds.width = 360;
  this.physics.world.bounds.height = 700;

  // add all level elements
  this.setupLevel();

  // collision detection
  this.physics.add.collider(this.player, this.platforms);
  this.physics.add.collider(this.goal, this.platforms);

  // enable cursor keys
  this.cursors = this.input.keyboard.createCursorKeys();

  this.input.on('pointerdown', function(pointer) {
    console.log(pointer.x, pointer.y);
  });
};

// executed on every frame
gameScene.update = function() {
  // are we on the ground?
```




```
let onGround = this.player.body.blocked.down ||
    this.player.body.touching.down;

// movement to the left
if (this.cursors.left.isDown) {
    this.player.body.setVelocityX(-this.playerSpeed);

    this.player.flipX = false;

    // play animation if none is playing
    if (onGround && !this.player.anims.isPlaying)
        this.player.anims.play('walking');
}

// movement to the right
else if (this.cursors.right.isDown) {
    this.player.body.setVelocityX(this.playerSpeed);

    this.player.flipX = true;

    // play animation if none is playing
    if (onGround && !this.player.anims.isPlaying)
        this.player.anims.play('walking');
} else {
    // make the player stop
    this.player.body.setVelocityX(0);

    // stop walking animation
    this.player.anims.stop('walking');

    // set default frame
    if (onGround)
        this.player.setFrame(3);
}

// handle jumping
if (onGround && (this.cursors.space.isDown || this.cursors.up.isDown)) {
    // give the player a velocity in Y
    this.player.body.setVelocityY(this.jumpSpeed);

    // stop the walking animation
    this.player.anims.stop('walking');

    // change frame
    this.player.setFrame(2);
}
};

// sets up all the elements in the level
gameScene.setupLevel = function() {

    // load json data
    this.levelData = this.cache.json.get('levelData');

    // create all the platforms
```



```
this.platforms = this.add.group();
for (let i = 0; i < this.levelData.platforms.length; i++) {
    let curr = this.levelData.platforms[i];

    let newObj;

    // create object
    if(curr.numTiles == 1) {
        // create sprite
        newObj = this.add.sprite(curr.x, curr.y, curr.key).setOrigin(0);
    }
    else {
        // create tilesprite
        let width = this.textures.get(curr.key).get(0).width;
        let height = this.textures.get(curr.key).get(0).height;
        newObj = this.add.tileSprite(curr.x, curr.y, curr.numTiles * width, height, curr.key).setOrigin(0);
    }

    // enable physics
    this.physics.add.existing(newObj, true);

    // add to the group
    this.platforms.add(newObj);
}

// create all the fire
this.fires = this.add.group();
for (let i = 0; i < this.levelData.fires.length; i++) {
    let curr = this.levelData.fires[i];

    let newObj = this.add.sprite(curr.x, curr.y, 'fire').setOrigin(0);

    // enable physics
    this.physics.add.existing(newObj);
    newObj.body.allowGravity = false;
    newObj.body.immovable = true;

    // play burning animation
    newObj.anims.play('burning');

    // add to the group
    this.fires.add(newObj);

    // this is for level creation
    newObj.setInteractive();
    this.input.setDraggable(newObj);
}

// for level creation
this.input.on('drag', function(pointer, gameObject, dragX, dragY){
    gameObject.x = dragX;
    gameObject.y = dragY;

    console.log(dragX, dragY);
```

```
});

// player
this.player = this.add.sprite(this.levelData.player.x, this.levelData.player.y, 'player', 3);
this.physics.add.existing(this.player);

// constraint player to the game bounds
this.player.body.setCollideWorldBounds(true);

// goal
this.goal = this.add.sprite(this.levelData.goal.x, this.levelData.goal.y, 'goal');
this.physics.add.existing(this.goal);

};

// our game's configuration
let config = {
  type: Phaser.AUTO,
  width: 360,
  height: 640,
  scene: gameScene,
  title: 'Monster Kong',
  pixelArt: false,
  physics: {
    default: 'arcade',
    arcade: {
      gravity: {
        y: 1000
      },
      debug: true
    }
  }
};

// create the game, and pass it the configuration
let game = new Phaser.Game(config);
```

Detecting Overlap

main.js

Found in Project/07 - Detecting Overlap/js

The code establishes a Phaser game with interactive level building and a camera following the player action. The game involves keyboard controls for player movement, animation for walking and burning effects, and collision/overlap detection which triggers restart of the game. For the level setup, positions of platforms and fires are draggable for an interactive design.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');
```

```
// some parameters for our scene
gameScene.init = function() {

    // player parameters
    this.playerSpeed = 150;
    this.jumpSpeed = -600;
};

// load asset files for our game
gameScene.preload = function() {

    // load images
    this.load.image('ground', 'assets/images/ground.png');
    this.load.image('platform', 'assets/images/platform.png');
    this.load.image('block', 'assets/images/block.png');
    this.load.image('goal', 'assets/images/gorilla3.png');
    this.load.image('barrel', 'assets/images/barrel.png');

    // load spritesheets
    this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
        frameWidth: 28,
        frameHeight: 30,
        margin: 1,
        spacing: 1
    });

    this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
        frameWidth: 20,
        frameHeight: 21,
        margin: 1,
        spacing: 1
    });

    this.load.json('levelData', 'assets/json/levelData.json');
};

// executed once, after assets were loaded
gameScene.create = function() {

    if(!this.anims.get('walking')) {
        // walking animation
        this.anims.create({
            key: 'walking',
            frames: this.anims.generateFrameNames('player', {
                frames: [0, 1, 2]
            }),
            frameRate: 12,
            yoyo: true,
            repeat: -1
        });
    }

    if(!this.anims.get('burning')) {
        // fire animation
    }
}
```



```
this.anims.create({
  key: 'burning',
  frames: this.anims.generateFrameNames('fire', {
    frames: [0, 1]
  }),
  frameRate: 4,
  repeat: -1
});
}

// add all level elements
this.setupLevel();

// world bounds
this.physics.world.bounds.width = 360;
this.physics.world.bounds.height = 700;

// camera bounds
this.cameras.main.setBounds(0, 0, 360, 700);
this.cameras.main.startFollow(this.player);

// collision detection
this.physics.add.collider([this.player, this.goal], this.platforms);

// overlap checks
this.physics.add.overlap(this.player, [this.fires, this.goal], this.restartGame, null, this);

// enable cursor keys
this.cursors = this.input.keyboard.createCursorKeys();

this.input.on('pointerdown', function(pointer) {
  console.log(pointer.x, pointer.y);
});
};

// executed on every frame
gameScene.update = function() {
  // are we on the ground?
  let onGround = this.player.body.blocked.down ||
    this.player.body.touching.down;

  // movement to the left
  if (this.cursors.left.isDown) {
    this.player.body.setVelocityX(-this.playerSpeed);

    this.player.flipX = false;

    // play animation if none is playing
    if (onGround && !this.player.anims.isPlaying)
      this.player.anims.play('walking');
  }

  // movement to the right
  else if (this.cursors.right.isDown) {
```



```
this.player.body.setVelocityX(this.playerSpeed);

this.player.flipX = true;

// play animation if none is playing
if (onGround && !this.player.anims.isPlaying)
    this.player.anims.play('walking');
} else {
    // make the player stop
    this.player.body.setVelocityX(0);

    // stop walking animation
    this.player.anims.stop('walking');

    // set default frame
    if (onGround)
        this.player.setFrame(3);
}

// handle jumping
if (onGround && (this.cursors.space.isDown || this.cursors.up.isDown)) {
    // give the player a velocity in Y
    this.player.body.setVelocityY(this.jumpSpeed);

    // stop the walking animation
    this.player.anims.stop('walking');

    // change frame
    this.player.setFrame(2);
}
};

// sets up all the elements in the level
gameScene.setupLevel = function() {

    // load json data
    this.levelData = this.cache.json.get('levelData');

    // create all the platforms
    this.platforms = this.physics.add.staticGroup();
    for (let i = 0; i < this.levelData.platforms.length; i++) {
        let curr = this.levelData.platforms[i];

        let newObj;

        // create object
        if(curr.numTiles == 1) {
            // create sprite
            newObj = this.add.sprite(curr.x, curr.y, curr.key).setOrigin(0);
        }
        else {
            // create tilesprite
            let width = this.textures.get(curr.key).get(0).width;
            let height = this.textures.get(curr.key).get(0).height;
            newObj = this.add.tileSprite(curr.x, curr.y, curr.numTiles * width, height, cu
```

```
rr.key).setOrigin(0);
    }

    // enable physics
    this.physics.add.existing(newObj, true);

    // add to the group
    this.platforms.add(newObj);
}

// create all the fire
this.fires = this.physics.add.group({
    allowGravity: false,
    immovable: true
});
for (let i = 0; i < this.levelData.fires.length; i++) {
    let curr = this.levelData.fires[i];

    let newObj = this.add.sprite(curr.x, curr.y, 'fire').setOrigin(0);

    // play burning animation
    newObj.anims.play('burning');

    // add to the group
    this.fires.add(newObj);

    // this is for level creation
    newObj.setInteractive();
    this.input.setDraggable(newObj);
}

// for level creation
this.input.on('drag', function(pointer, gameObject, dragX, dragY){
    gameObject.x = dragX;
    gameObject.y = dragY;

    console.log(dragX, dragY);

});

// player
this.player = this.add.sprite(this.levelData.player.x, this.levelData.player.y, 'player', 3);
this.physics.add.existing(this.player);

// constraint player to the game bounds
this.player.body.setCollideWorldBounds(true);

// goal
this.goal = this.add.sprite(this.levelData.goal.x, this.levelData.goal.y, 'goal');
this.physics.add.existing(this.goal);
};

// restart game (game over + you won!)
gameScene.restartGame = function(sourceSprite, targetSprite){
```

```
// fade out
this.cameras.main.fade(500);

// when fade out completes, restart scene
this.cameras.main.on('camerafadeoutcomplete', function(camera, effect){
  // restart the scene
  this.scene.restart();
}, this);
};

// our game's configuration
let config = {
  type: Phaser.AUTO,
  width: 360,
  height: 640,
  scene: gameScene,
  title: 'Monster Kong',
  pixelArt: false,
  physics: {
    default: 'arcade',
    arcade: {
      gravity: {
        y: 1000
      },
      debug: true
    }
  }
};

// create the game, and pass it the configuration
let game = new Phaser.Game(config);
```

Generating Barrels

main.js

Found in Project/08 - Generating Barrels/js

This code example is a Phaser-based game called 'Monster Kong.' It initially sets up the game scene, loads assets, and establishes player movement parameters. Then, it uses JSON data to construct the level, adds game physics and colliders, and organizes animations. It handles user player movement and interactions, implements a spawner for enemy barrels, and has restart functionality upon game over.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// some parameters for our scene
gameScene.init = function() {

  // player parameters
  this.playerSpeed = 150;
  this.jumpSpeed = -600;
```



```
};

// load asset files for our game
gameScene.preload = function() {

    // load images
    this.load.image('ground', 'assets/images/ground.png');
    this.load.image('platform', 'assets/images/platform.png');
    this.load.image('block', 'assets/images/block.png');
    this.load.image('goal', 'assets/images/gorilla3.png');
    this.load.image('barrel', 'assets/images/barrel.png');

    // load spritesheets
    this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
        frameWidth: 28,
        frameHeight: 30,
        margin: 1,
        spacing: 1
    });

    this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
        frameWidth: 20,
        frameHeight: 21,
        margin: 1,
        spacing: 1
    });

    this.load.json('levelData', 'assets/json/levelData.json');
};

// executed once, after assets were loaded
gameScene.create = function() {

    if(!this.anims.get('walking')) {
        // walking animation
        this.anims.create({
            key: 'walking',
            frames: this.anims.generateFrameNames('player', {
                frames: [0, 1, 2]
            }),
            frameRate: 12,
            yoyo: true,
            repeat: -1
        });
    }

    if(!this.anims.get('burning')) {
        // fire animation
        this.anims.create({
            key: 'burning',
            frames: this.anims.generateFrameNames('fire', {
                frames: [0, 1]
            }),
            frameRate: 4,
            repeat: -1
        });
    }
};
```

```
    });
  }

  // add all level elements
  this.setupLevel();

  // initiate barrel spawner
  this.setupSpawner();

  // collision detection
  this.physics.add.collider([this.player, this.goal, this.barrels], this.platforms);

  // overlap checks
  this.physics.add.overlap(this.player, [this.fires, this.goal, this.barrels], this.r
  estartGame, null, this);

  // enable cursor keys
  this.cursors = this.input.keyboard.createCursorKeys();

  this.input.on('pointerdown', function(pointer) {
    console.log(pointer.x, pointer.y);
  });
};

// executed on every frame
gameScene.update = function() {
  // are we on the ground?
  let onGround = this.player.body.blocked.down || this.player.body.touching.down;

  // movement to the left
  if (this.cursors.left.isDown) {
    this.player.body.setVelocityX(-this.playerSpeed);

    this.player.flipX = false;

    // play animation if none is playing
    if (onGround && !this.player.anims.isPlaying)
      this.player.anims.play('walking');
  }

  // movement to the right
  else if (this.cursors.right.isDown) {
    this.player.body.setVelocityX(this.playerSpeed);

    this.player.flipX = true;

    // play animation if none is playing
    if (onGround && !this.player.anims.isPlaying)
      this.player.anims.play('walking');
  } else {
    // make the player stop
    this.player.body.setVelocityX(0);

    // stop walking animation
    this.player.anims.stop('walking');
```



```
// set default frame
if (onGround)
    this.player.setFrame(3);
}

// handle jumping
if (onGround && (this.cursors.space.isDown || this.cursors.up.isDown)) {
    // give the player a velocity in Y
    this.player.body.setVelocityY(this.jumpSpeed);

    // stop the walking animation
    this.player.anims.stop('walking');

    // change frame
    this.player.setFrame(2);
}
};

// sets up all the elements in the level
gameScene.setupLevel = function() {

    // load json data
    this.levelData = this.cache.json.get('levelData');

    // world bounds
    this.physics.world.bounds.width = this.levelData.world.width;
    this.physics.world.bounds.height = this.levelData.world.height;

    // create all the platforms
    this.platforms = this.physics.add.staticGroup();
    for (let i = 0; i < this.levelData.platforms.length; i++) {
        let curr = this.levelData.platforms[i];

        let newObj;

        // create object
        if(curr.numTiles == 1) {
            // create sprite
            newObj = this.add.sprite(curr.x, curr.y, curr.key).setOrigin(0);
        }
        else {
            // create tilesprite
            let width = this.textures.get(curr.key).get(0).width;
            let height = this.textures.get(curr.key).get(0).height;
            newObj = this.add.tileSprite(curr.x, curr.y, curr.numTiles * width, height, curr.key).setOrigin(0);
        }

        // enable physics
        this.physics.add.existing(newObj, true);

        // add to the group
        this.platforms.add(newObj);
    }
}
```



```
// create all the fire
this.fires = this.physics.add.group({
  allowGravity: false,
  immovable: true
});
for (let i = 0; i < this.levelData.fires.length; i++) {
  let curr = this.levelData.fires[i];

  let newObj = this.add.sprite(curr.x, curr.y, 'fire').setOrigin(0);

  // play burning animation
  newObj.anims.play('burning');

  // add to the group
  this.fires.add(newObj);

  // this is for level creation
  newObj.setInteractive();
  this.input.setDraggable(newObj);
}

// for level creation
this.input.on('drag', function(pointer, gameObject, dragX, dragY){
  gameObject.x = dragX;
  gameObject.y = dragY;

  console.log(dragX, dragY);

});

// player
this.player = this.add.sprite(this.levelData.player.x, this.levelData.player.y, 'player', 3);
this.physics.add.existing(this.player);

// constraint player to the game bounds
this.player.body.setCollideWorldBounds(true);

// camera bounds
this.cameras.main.setBounds(0, 0, this.levelData.world.width, this.levelData.world.height);
this.cameras.main.startFollow(this.player);

// goal
this.goal = this.add.sprite(this.levelData.goal.x, this.levelData.goal.y, 'goal');
this.physics.add.existing(this.goal);
};

// restart game (game over + you won!)
gameScene.restartGame = function(sourceSprite, targetSprite){
  // fade out
  this.cameras.main.fade(500);
}
```



```
// when fade out completes, restart scene
this.cameras.main.on('camerafadeoutcomplete', function(camera, effect){
  // restart the scene
  this.scene.restart();
}, this);
};

// generation of barrels
gameScene.setupSpawner = function(){
  // barrel group
  this.barrels = this.physics.add.group({
    bounceY: 0.1,
    bounceX: 1,
    collideworldBounds: true
  });

  // spawn barrels
  let spawningEvent = this.time.addEvent({
    delay: this.levelData.spawner.interval,
    loop: true,
    callbackScope: this,
    callback: function(){
      // create a barrel
      let barrel = this.barrels.create(this.goal.x, this.goal.y, 'barrel');

      // set properties
      barrel.setVelocityX(this.levelData.spawner.speed);

      // lifespan
      this.time.addEvent({
        delay: this.levelData.spawner.lifespan,
        repeat: 0,
        callbackScope: this,
        callback: function(){
          barrel.destroy();
        }
      });
    }
  });
};

// our game's configuration
let config = {
  type: Phaser.AUTO,
  width: 360,
  height: 640,
  scene: gameScene,
  title: 'Monster Kong',
  pixelArt: false,
  physics: {
    default: 'arcade',
    arcade: {
      gravity: {
        y: 1000
      },
    },
  },
};
```

```
        debug: true
    }
}
};

// create the game, and pass it the configuration
let game = new Phaser.Game(config);
```

Pool of Objects

main.js

Found in Project/09 - Pool of Objects/js

This code creates a Phaser-based game 'Monster Kong' where a player character moves and jumps around with set parameters. It loads images and sprite sheets, sets up the game level with platforms and fires, and defines player responses to key commands. It also includes collision and overlap checks, barrel spawner, and a game restart mechanism.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// some parameters for our scene
gameScene.init = function() {

    // player parameters
    this.playerSpeed = 150;
    this.jumpSpeed = -600;
};

// load asset files for our game
gameScene.preload = function() {

    // load images
    this.load.image('ground', 'assets/images/ground.png');
    this.load.image('platform', 'assets/images/platform.png');
    this.load.image('block', 'assets/images/block.png');
    this.load.image('goal', 'assets/images/gorilla3.png');
    this.load.image('barrel', 'assets/images/barrel.png');

    // load spritesheets
    this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
        frameWidth: 28,
        frameHeight: 30,
        margin: 1,
        spacing: 1
    });

    this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
        frameWidth: 20,
        frameHeight: 21,
        margin: 1,
        spacing: 1
    });
};
```

```
});

this.load.json('levelData', 'assets/json/levelData.json');
};

// executed once, after assets were loaded
gameScene.create = function() {

    if(!this.anims.get('walking')) {
        // walking animation
        this.anims.create({
            key: 'walking',
            frames: this.anims.generateFrameNames('player', {
                frames: [0, 1, 2]
            }),
            frameRate: 12,
            yoyo: true,
            repeat: -1
        });
    }

    if(!this.anims.get('burning')) {
        // fire animation
        this.anims.create({
            key: 'burning',
            frames: this.anims.generateFrameNames('fire', {
                frames: [0, 1]
            }),
            frameRate: 4,
            repeat: -1
        });
    }

    // add all level elements
    this.setupLevel();

    // initiate barrel spawner
    this.setupSpawner();

    // collision detection
    this.physics.add.collider([this.player, this.goal, this.barrels], this.platforms);

    // overlap checks
    this.physics.add.overlap(this.player, [this.fires, this.goal, this.barrels], this.restartGame, null, this);

    // enable cursor keys
    this.cursors = this.input.keyboard.createCursorKeys();

    this.input.on('pointerdown', function(pointer) {
        console.log(pointer.x, pointer.y);
    });
};

// executed on every frame
```

```
gameScene.update = function() {
  // are we on the ground?
  let onGround = this.player.body.blocked.down || this.player.body.touching.down;

  // movement to the left
  if (this.cursors.left.isDown) {
    this.player.body.setVelocityX(-this.playerSpeed);

    this.player.flipX = false;

    // play animation if none is playing
    if (onGround && !this.player.anims.isPlaying)
      this.player.anims.play('walking');
  }

  // movement to the right
  else if (this.cursors.right.isDown) {
    this.player.body.setVelocityX(this.playerSpeed);

    this.player.flipX = true;

    // play animation if none is playing
    if (onGround && !this.player.anims.isPlaying)
      this.player.anims.play('walking');
  } else {
    // make the player stop
    this.player.body.setVelocityX(0);

    // stop walking animation
    this.player.anims.stop('walking');

    // set default frame
    if (onGround)
      this.player.setFrame(3);
  }

  // handle jumping
  if (onGround && (this.cursors.space.isDown || this.cursors.up.isDown)) {
    // give the player a velocity in Y
    this.player.body.setVelocityY(this.jumpSpeed);

    // stop the walking animation
    this.player.anims.stop('walking');

    // change frame
    this.player.setFrame(2);
  }
};

// sets up all the elements in the level
gameScene.setupLevel = function() {

  // load json data
  this.levelData = this.cache.json.get('levelData');
```



```
// world bounds
this.physics.world.bounds.width = this.levelData.world.width;
this.physics.world.bounds.height = this.levelData.world.height;

// create all the platforms
this.platforms = this.physics.add.staticGroup();
for (let i = 0; i < this.levelData.platforms.length; i++) {
    let curr = this.levelData.platforms[i];

    let newObj;

    // create object
    if(curr.numTiles == 1) {
        // create sprite
        newObj = this.add.sprite(curr.x, curr.y, curr.key).setOrigin(0);
    }
    else {
        // create tilesprite
        let width = this.textures.get(curr.key).get(0).width;
        let height = this.textures.get(curr.key).get(0).height;
        newObj = this.add.tileSprite(curr.x, curr.y, curr.numTiles * width, height, curr.key).setOrigin(0);
    }

    // enable physics
    this.physics.add.existing(newObj, true);

    // add to the group
    this.platforms.add(newObj);
}

// create all the fire
this.fires = this.physics.add.group({
    allowGravity: false,
    immovable: true
});
for (let i = 0; i < this.levelData.fires.length; i++) {
    let curr = this.levelData.fires[i];

    let newObj = this.add.sprite(curr.x, curr.y, 'fire').setOrigin(0);

    // play burning animation
    newObj.anims.play('burning');

    // add to the group
    this.fires.add(newObj);

    // this is for level creation
    newObj.setInteractive();
    this.input.setDraggable(newObj);
}

// for level creation
this.input.on('drag', function(pointer, gameObject, dragX, dragY){
    gameObject.x = dragX;
```

```
gameObject.y = dragY;

console.log(dragX, dragY);

});

// player
this.player = this.add.sprite(this.levelData.player.x, this.levelData.player.y, 'player', 3);
this.physics.add.existing(this.player);

// constraint player to the game bounds
this.player.body.setCollideWorldBounds(true);

// camera bounds
this.cameras.main.setBounds(0, 0, this.levelData.world.width, this.levelData.world.height);
this.cameras.main.startFollow(this.player);

// goal
this.goal = this.add.sprite(this.levelData.goal.x, this.levelData.goal.y, 'goal');
this.physics.add.existing(this.goal);
};

// restart game (game over + you won!)
gameScene.restartGame = function(sourceSprite, targetSprite){
    // fade out
    this.cameras.main.fade(500);

    // when fade out completes, restart scene
    this.cameras.main.on('camerafadeoutcomplete', function(camera, effect){
        // restart the scene
        this.scene.restart();
    }, this);
};

// generation of barrels
gameScene.setupSpawner = function(){
    // barrel group
    this.barrels = this.physics.add.group({
        bounceY: 0.1,
        bounceX: 1,
        collideWorldBounds: true
    });

    // spawn barrels
    let spawningEvent = this.time.addEvent({
        delay: this.levelData.spawner.interval,
        loop: true,
        callbackScope: this,
        callback: function(){
            // create a barrel
            let barrel = this.barrels.get(this.goal.x, this.goal.y, 'barrel');
```



```
// reactivate
barrel.setActive(true);
barrel.setVisible(true);
barrel.body.enable = true;

// set properties
barrel.setVelocityX(this.levelData.spawner.speed);

//console.log(this.barrels.getChildren().length);

// lifespan
this.time.addEvent({
  delay: this.levelData.spawner.lifespan,
  repeat: 0,
  callbackScope: this,
  callback: function(){
    this.barrels.killAndHide(barrel);
    barrel.body.enable = false;
  }
});
});
};

// our game's configuration
let config = {
  type: Phaser.AUTO,
  width: 360,
  height: 640,
  scene: gameScene,
  title: 'Monster Kong',
  pixelArt: false,
  physics: {
    default: 'arcade',
    arcade: {
      gravity: {
        y: 1000
      },
      debug: false
    }
  }
};

// create the game, and pass it the configuration
let game = new Phaser.Game(config);
```

levelData.json

Found in Project/09 - Pool of Objects/assets/json

This is a JSON representation of the level data for 'Monster Kong' game. It defines the parameters of the game world, the platform layouts, the starting positions of the player and goal, the spawner settings, and the locations of the fires.

```
{
  "world": {
    "width": 360,
    "height": 700
  },
  "spawner": {
    "interval": 3000,
    "speed": 80,
    "lifespan": 30000
  },
  "player": {
    "x": 190,
    "y": 538
  },
  "goal": {
    "x": 30,
    "y": 65
  },
  "platforms": [{
    "x": 72,
    "y": 450,
    "numTiles": 6,
    "key": "block"
  },
  {
    "x": 0,
    "y": 330,
    "numTiles": 8,
    "key": "block"
  },
  {
    "x": 72,
    "y": 210,
    "numTiles": 8,
    "key": "block"
  },
  {
    "x": 0,
    "y": 90,
    "numTiles": 7,
    "key": "block"
  },
  {
    "x": 0,
    "y": 560,
    "numTiles": 1,
    "key": "ground"
  }
],
  "fires": [
    {
      "x": 12.9,
      "y": 538.4
    },
    {
```

```
        "x": 328,  
        "y": 188  
    },  
    {  
        "x": 68.3,  
        "y": 307  
    },  
    {  
        "x": 87.7,  
        "y": 427  
    }  
]  
}
```